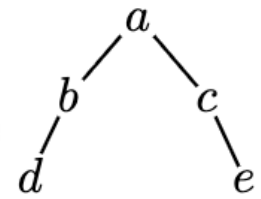
Comp90038
Tutorial 7:
Week 8

Today's Tutorial Flow

- Tutorial Questions
  - Divide and conquer: Q50 (practice)
  - BST: Q51, 52, 53, 54
  - Max-heap: Q56, 57, 58, 59

Tutorial Questions

51. Traverse 

in preorder, inorder, and postorder.

Binary Tree Traversal

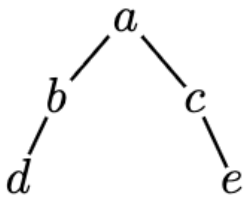
THE UNIVERSITY OF MELBOURNE

- **Preorder** traversal visits the root, then the left subtree, and finally the right subtree.
- **Inorder** traversal visits the left subtree, then the root, and finally the right subtree.
- **Postorder** traversal visits the left subtree, the right subtree, and finally the root.

```
procedure PreorderTraverse(T)
    if T ≠ null then
        visit T.root
        PreorderTraverse(T.left)
        PreorderTraverse(T.right)
procedure InorderTraverse(T)
    if T ≠ null then
        InorderTraverse(T.left)
        visit T.root
        InorderTraverse(T.right)

procedure PostorderTraverse(T)
    if T ≠ null then
        PostorderTraverse(T.left)
        PostorderTraverse(T.right)
        visit T.root
```

Tutorial Questions

51. Traverse 

in preorder, inorder, and postorder.

Binary Tree Traversal

THE UNIVERSITY OF MELBOURNE

- **Preorder** traversal visits the root, then the left subtree, and finally the right subtree.
- **Inorder** traversal visits the left subtree, then the root, and finally the right subtree.
- **Postorder** traversal visits the left subtree, the right subtree, and finally the root.

Answer: Preorder: a–b–d–c–e. Inorder: d–b–a–c–e. Postorder: d–b–e–c–a.

Tutorial Questions

52. A certain binary tree yields a, b, c, d, e when traversed preorder, and it yields the same sequence when traversed inorder. What does the binary tree look like?

Binary Tree Traversal



- **Preorder** traversal visits the root, then the left subtree, and finally the right subtree.
- **Inorder** traversal visits the left subtree, then the root, and finally the right subtree.
- **Postorder** traversal visits the left subtree, the right subtree, and finally the root.

Tutorial Questions

52. A certain binary tree yields a, b, c, d, e when traversed preorder, and it yields the same sequence when traversed inorder. What does the binary tree look like?

Binary Tree Traversal



- **Preorder** traversal visits the root, then the left subtree, and finally the right subtree.
- **Inorder** traversal visits the left subtree, then the root, and finally the right subtree.
- **Postorder** traversal visits the left subtree, the right subtree, and finally the root.

Answer: It is a stick, with nodes $a \cdots e$, each parent having only a right-child.

Tutorial Questions

53. A certain binary tree yields 14, 83, 63, 42, 19, 27, 74, 99, 51, 37 when traversed preorder and 63, 83, 19, 42, 14, 27, 99, 51, 74, 37 when traversed inorder. Which sequence does it yield when traversed postorder?

Binary Tree Traversal



- **Preorder** traversal visits the root, then the left subtree, and finally the right subtree.
- **Inorder** traversal visits the left subtree, then the root, and finally the right subtree.
- **Postorder** traversal visits the left subtree, the right subtree, and finally the root.

Tutorial Questions

53. A certain binary tree yields 14, 83, 63, 42, 19, 27, 74, 99, 51, 37 when traversed preorder and 63, 83, 19, 42, 14, 27, 99, 51, 74, 37 when traversed inorder. Which sequence does it yield when traversed postorder?

Binary Tree Traversal

THE UNIVERSITY OF MELBOURNE

- **Preorder** traversal visits the root, then the left subtree, and finally the right subtree.
- **Inorder** traversal visits the left subtree, then the root, and finally the right subtree.
- **Postorder** traversal visits the left subtree, the right subtree, and finally the root.

Answer: Postorder yields 63, 19, 42, 83, 51, 99, 37, 74, 27, 14. To see this, construct the binary tree from the preorder and inorder sequences (this is best handled as a recursive process).

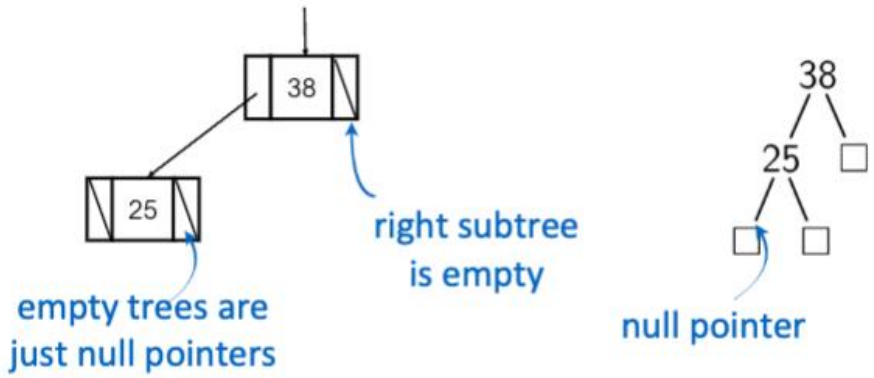
Tutorial Questions

54. The following algorithm was designed to compute the number of leaves in a binary tree T. We denote the empty tree by *null*.

```
function LeafCount(T)
    if T = null then
        return 0
    else
        return LeafCount(T.left) + LeafCount(T.right)
```

Empty Nodes

THE UNIVERSITY OF MELBOURNE



Tutorial Questions

54. The following algorithm was designed to compute the number of leaves in a binary tree T. We denote the empty tree by *null*.

```
function LeafCount(T)
    if T = null then
        return 0
    else
        return LeafCount(T.left) + LeafCount(T.right)
```

Answer: We need to check for the case of a single-node tree (or sub-tree):

```
function LeafCount(T)
    if T = null then
        return 0
    else
        if T.left = null and T.right = null then
            return 1
        else
            return LeafCount(T.left) + LeafCount(T.right)
```

Tutorial Questions

Priority Queues

A priority queue is a set (or pool) of elements.

An element is injected into the priority queue together with a priority (often the key value itself) and elements are ejected according to priority.

The Priority Queue

As an abstract data type, the priority queue supports the following operations on a "pool" of elements (ordered by some linear order):

- find an item with maximal priority
- insert a new item with associated priority
- test whether a priority queue is empty
- eject the largest element

Some Uses of Priority Queues

- Job scheduling done by your operating system. The OS will usually have a notion of "importance" of different jobs.
- Traffic management (e.g., arrival times of network requests)
- (Discrete event) simulation of complex systems (like traffic, or weather). Here priorities are typically event times.

Many sophisticated algorithms make essential use of priority queues (Huffman encoding and many shortest-path algorithms, for example).

Tutorial Questions



Implementations of the Priority Queues

Assume priority = key.

	INJECT(e)	EJECT()
Unsorted array or list		
Sorted array or list		
Heap	$O(\log n)$	$O(\log n)$

Very useful data structure:
priority queues but also many algorithms.

Heap: partially ordered binary tree.

Easily maintained as a complete tree.

Conveniently represented in arrays.

Tutorial Questions

A heap is a complete binary tree which satisfies the heap condition:

Each child has a priority (key) not greater than its parent's.

This guarantees that the root of the tree is a maximal element.

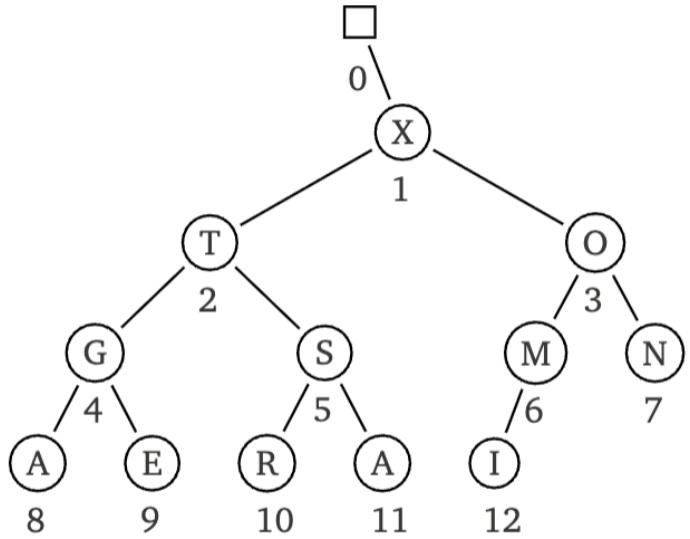
(Sometimes we talk about this as a max-heap—one can equally well have min-heaps, in which each child is no smaller than its parent.)

Heaps as Arrays

Elements in level-order placed in array H. (complete tree!)

Children of node i are nodes $2i$ and $2i+1$.

Heap condition easy: For all $i \in \{0, 1, \ldots, n\}$, must have $H[i] \leq H[i/2]$.



H:		X	T	O	G	S	M	N	A	E	R	A	I
	0	1	2	3	4	5	6	7	8	9	10	11	12

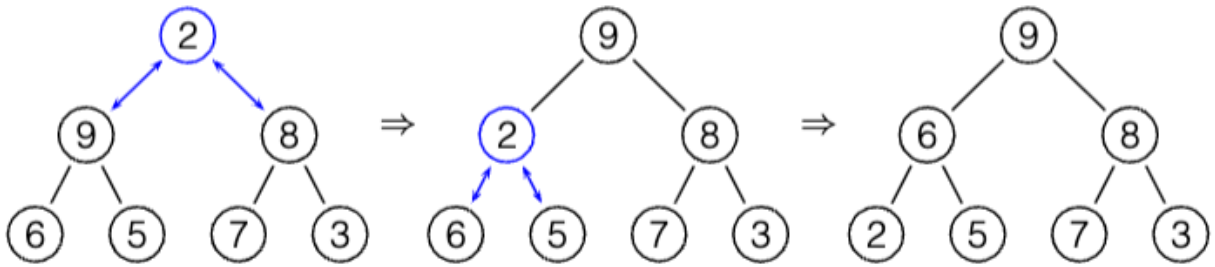
Tutorial Questions

56. Make a max-heap out of the keys A, L, G, O, R, I, T, H, M, using the bottom-up algorithm.

Turning $H[1]\ldots H[n]$ into a Heap

```
for i ← ⌊n/2⌋ downto 1 do
    k ← i
    v ← H[k]
    heap ← False
    while not heap and 2 × k ≤ n do
        j ← 2 × k                          ▷ j is k's left child
        if j < n then
            if H[j] < H[j + 1] then
                j ← j + 1                  ▷ j is k's largest child
        if v ≥ H[j] then
            heap ← True
        else                               ▷ Promote H[j]
            H[k] ← H[j]
            k ← j
    H[k] ← v
```

Whenever a parent is found to be out of order, let it "sift down" until both children are smaller:

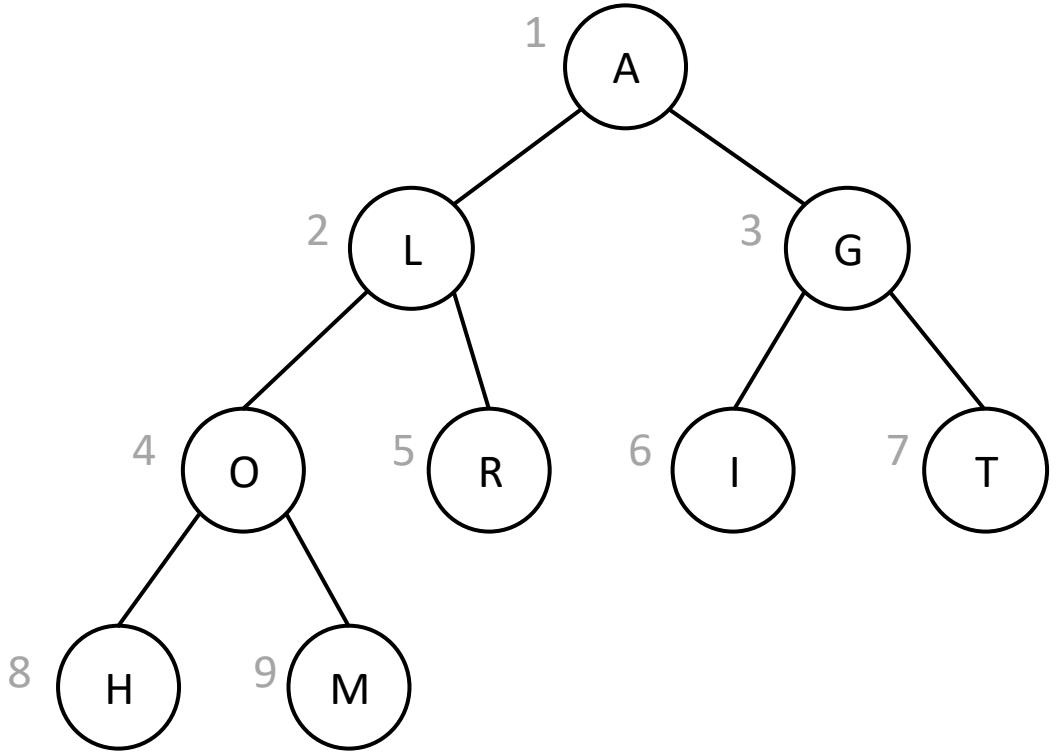


Problem 3. Make a max-heap out of the keys A, L, G, O, R, I, T, H, M, using the bottom-up algorithm.

- for $i = n/2, \ldots, 1$, invoke *bubble-down*(i);

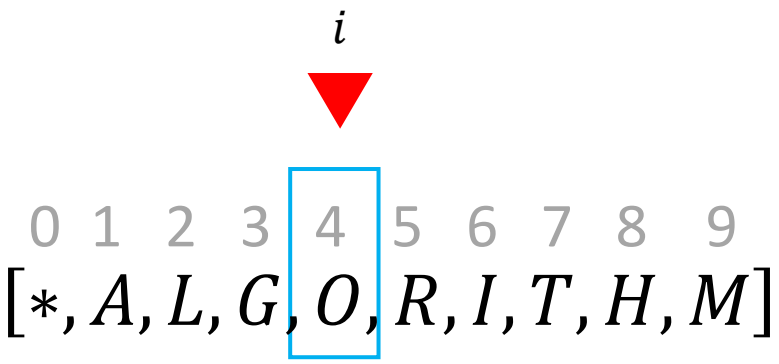
0 1 2 3 4 5 6 7 8 9

$[*, A, L, G, O, R, I, T, H, M]$

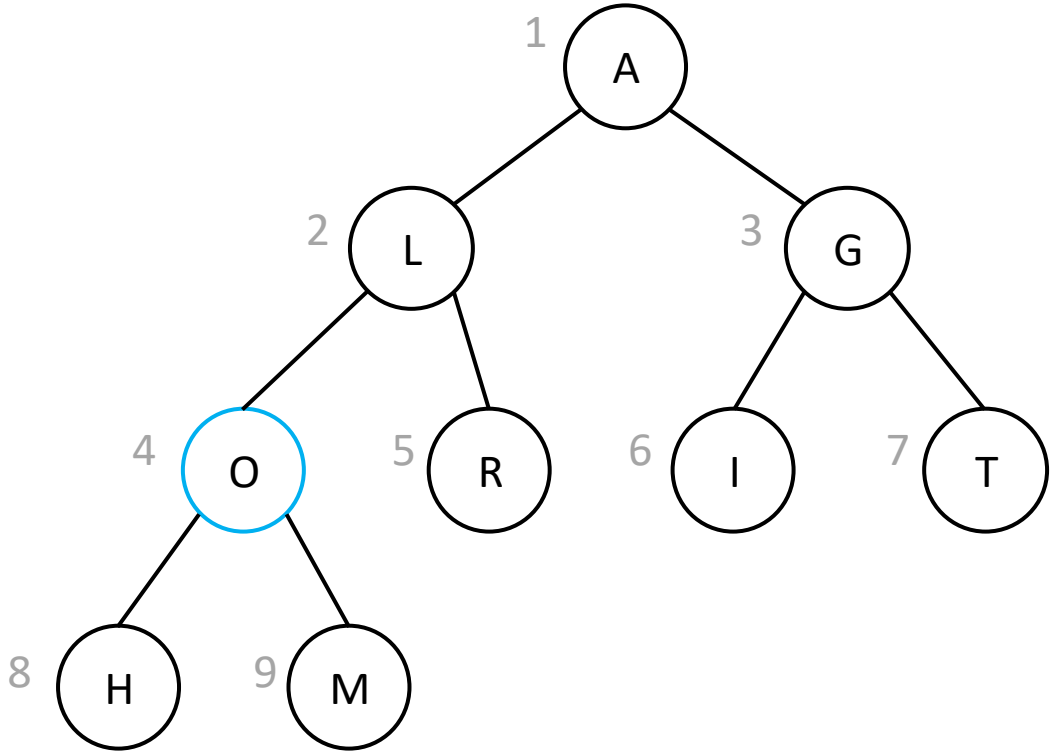


Problem 3. Make a max-heap out of the keys A, L, G, O, R, I, T, H, M, using the bottom-up algorithm.

- for $i = n/2, \ldots, 1$, invoke *bubble-down*(i);



bubble-down(4)



Problem 3. Make a max-heap out of the keys A, L, G, O, R, I, T, H, M, using the bottom-up algorithm.

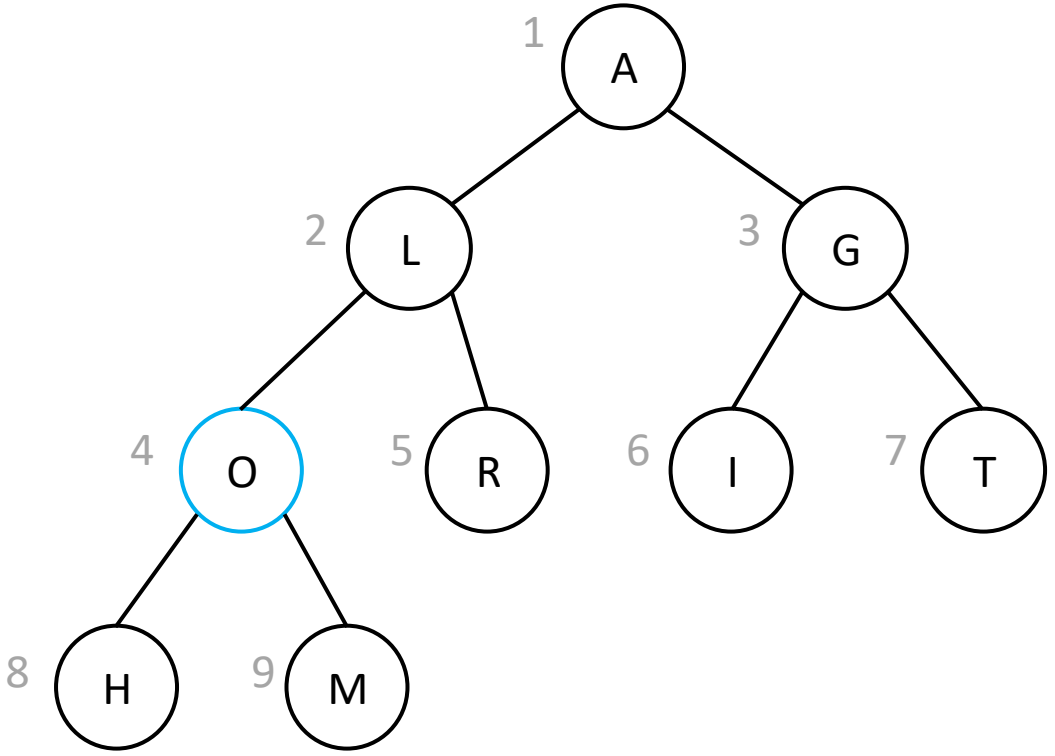
- for $i = n/2, \ldots, 1$, invoke *bubble-down*(i);

i

0	1	2	3	4	5	6	7	8	9
$*$	A	L	G	O	R	I	T	H	M

bubble-down(4)

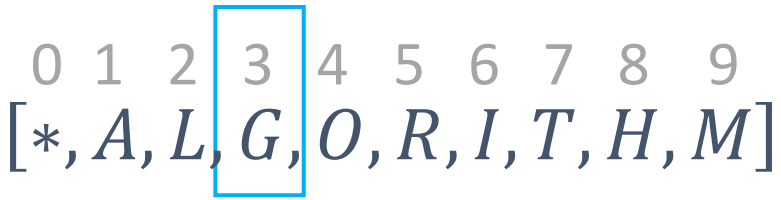
O is already the max of $\{O, H, M\}$.



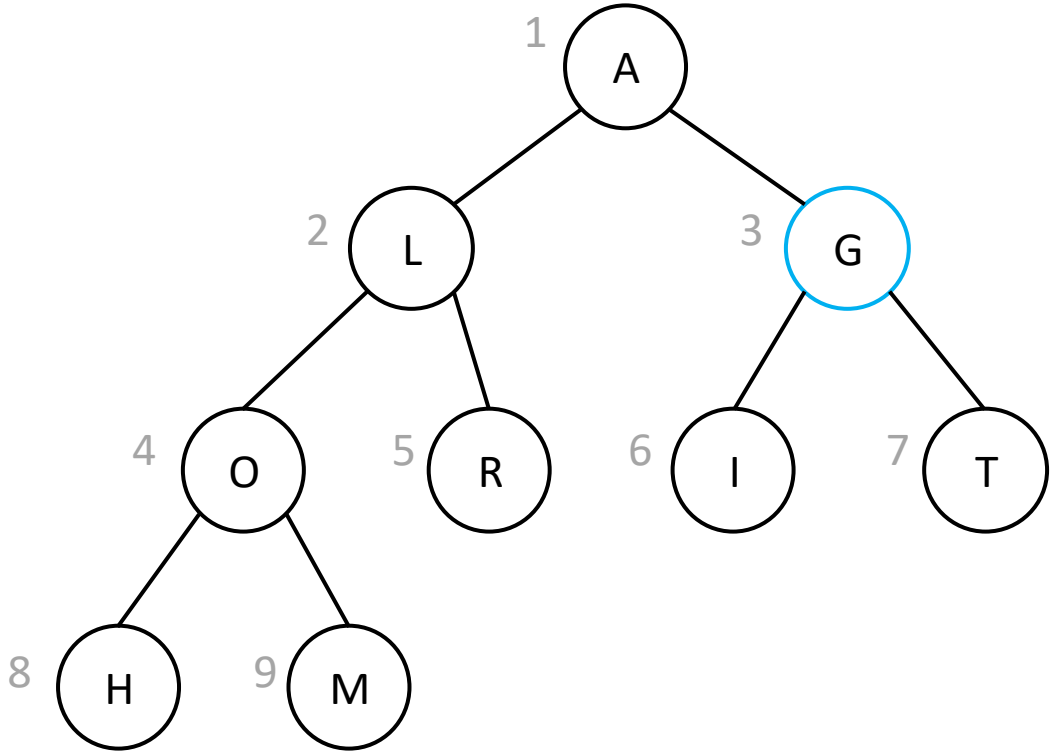
Problem 3. Make a max-heap out of the keys A, L, G, O, R, I, T, H, M, using the bottom-up algorithm.

- for $i = n/2, \ldots, 1$, invoke *bubble-down*(i);

i



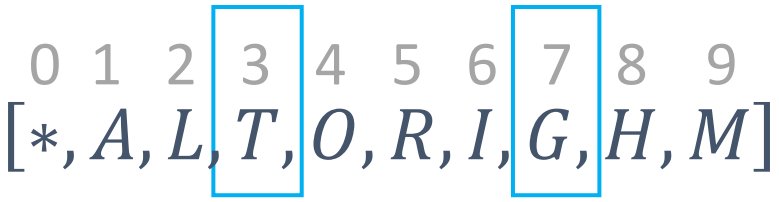
bubble-down(3)



Problem 3. Make a max-heap out of the keys A, L, G, O, R, I, T, H, M, using the bottom-up algorithm.

- for $i = n/2, \ldots, 1$, invoke *bubble-down*(i);

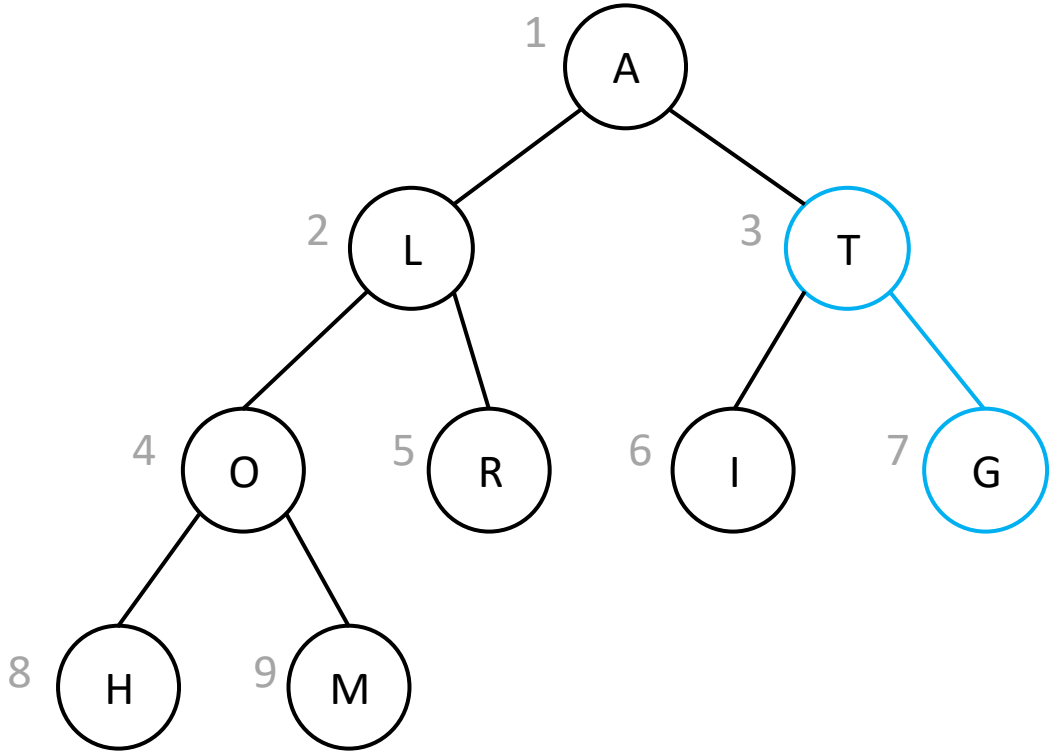
i



bubble-down(3)

T is the max out of $\{G, I, T\}$.

So swap G and T.



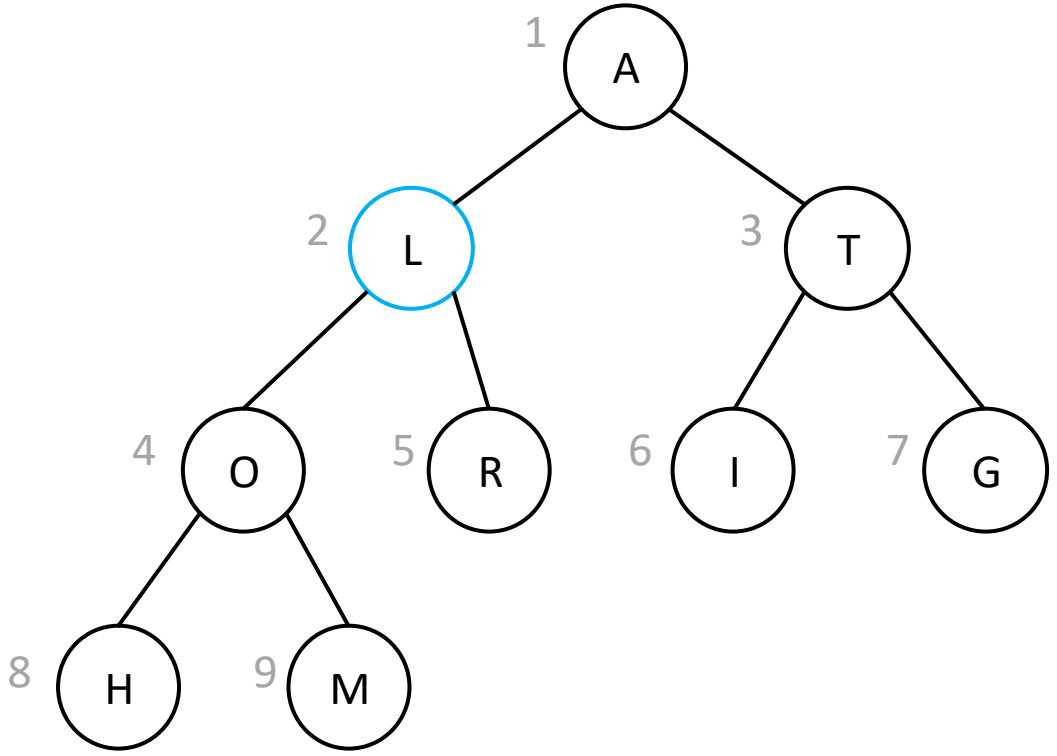
Problem 3. Make a max-heap out of the keys A, L, G, O, R, I, T, H, M, using the bottom-up algorithm.

- for $i = n/2, \ldots, 1$, invoke *bubble-down*(i);

i

0	1	2	3	4	5	6	7	8	9
$*$	A	L	T	O	R	I	G	H	M

bubble-down(2)

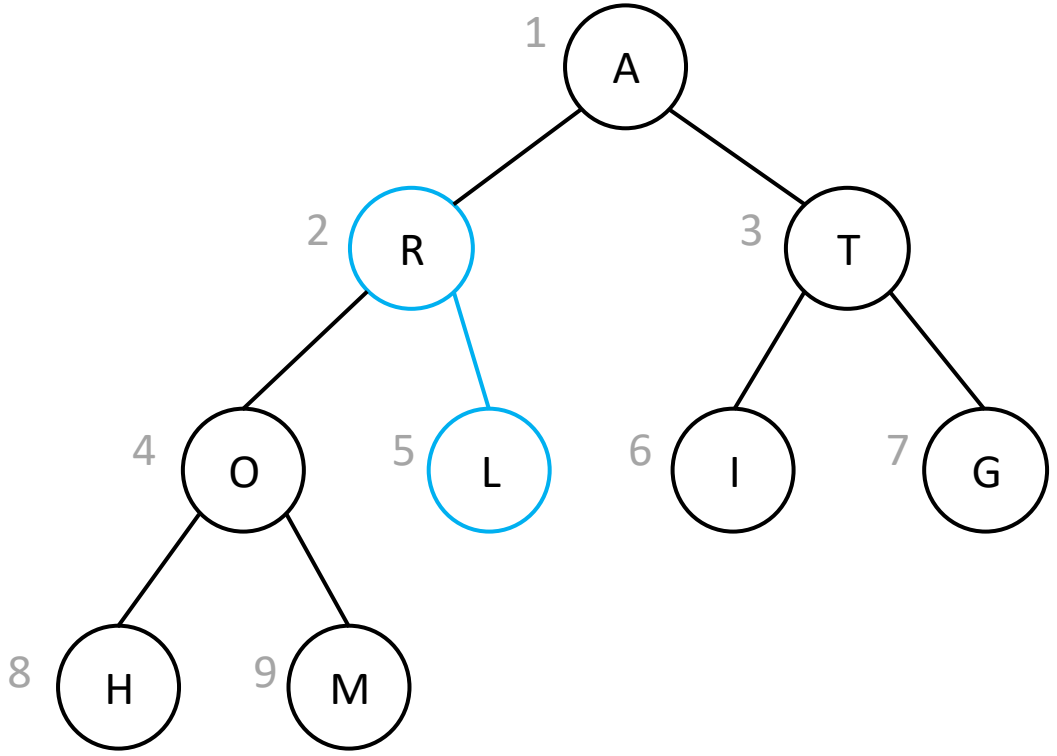


Problem 3. Make a max-heap out of the keys A, L, G, O, R, I, T, H, M, using the bottom-up algorithm.

- for $i = n/2, \ldots, 1$, invoke *bubble-down*(i);

i

0	1	2	3	4	5	6	7	8	9
$*$	A	R	T	O	L	I	G	H	M



bubble-down(2)

R is the max out of $\{L, O, R\}$.

So swap L and R.

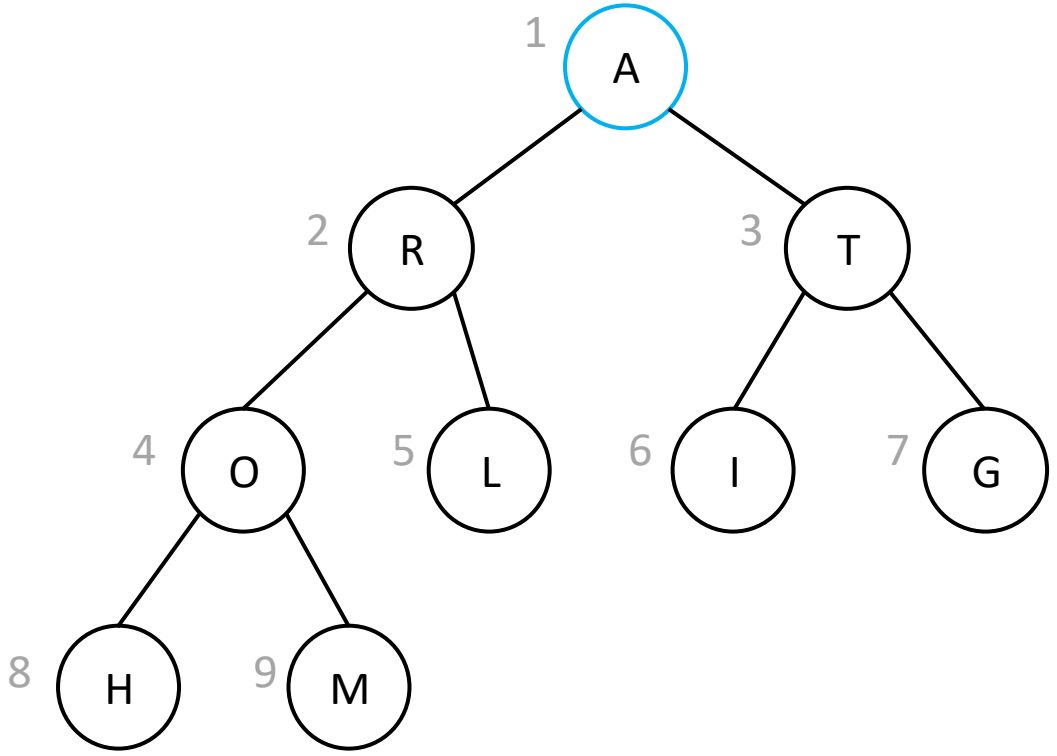
Problem 3. Make a max-heap out of the keys A, L, G, O, R, I, T, H, M, using the bottom-up algorithm.

- for $i = n/2, \ldots, 1$, invoke *bubble-down*(i);

i

0	1	2	3	4	5	6	7	8	9
*	A	R	T	O	L	I	G	H	M

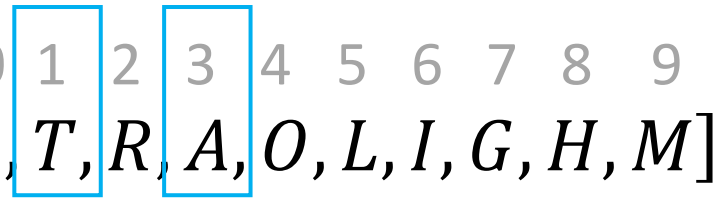
bubble-down(1)

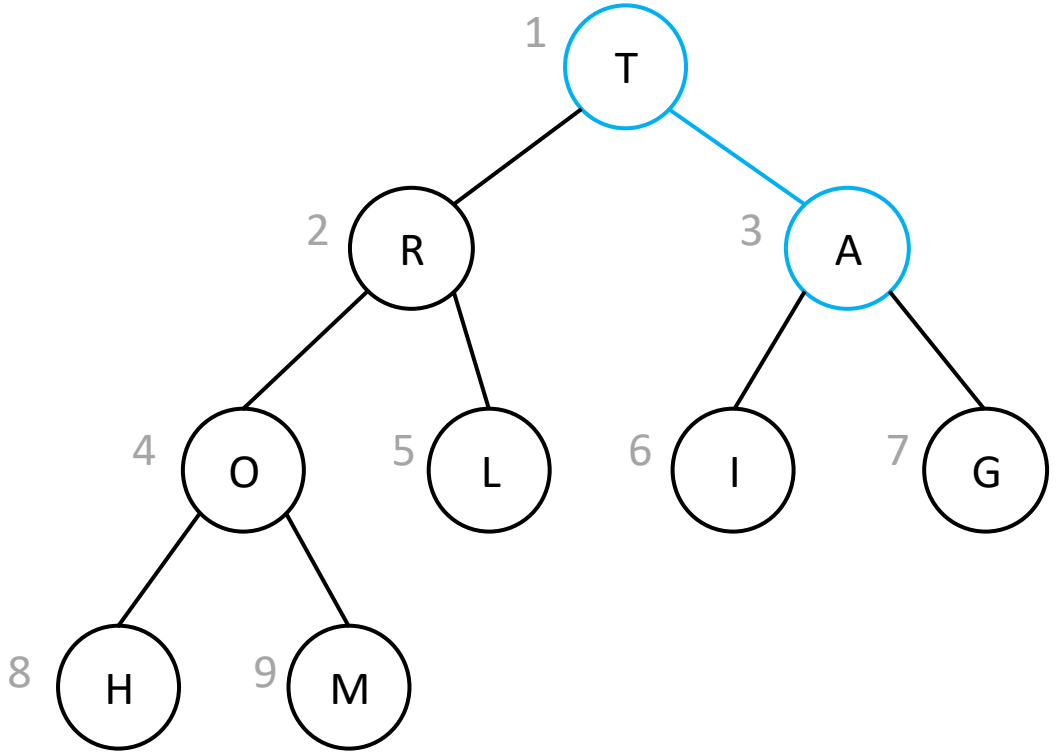


Problem 3. Make a max-heap out of the keys A, L, G, O, R, I, T, H, M, using the bottom-up algorithm.

- for $i = n/2, \ldots, 1$, invoke *bubble-down*(i);

i





bubble-down(1)

T is the max out of $\{A, R, T\}$.

So swap A and T.

Problem 3. Make a max-heap out of the keys A, L, G, O, R, I, T, H, M, using the bottom-up algorithm.

- for $i = n/2, \ldots, 1$, invoke *bubble-down*(i);

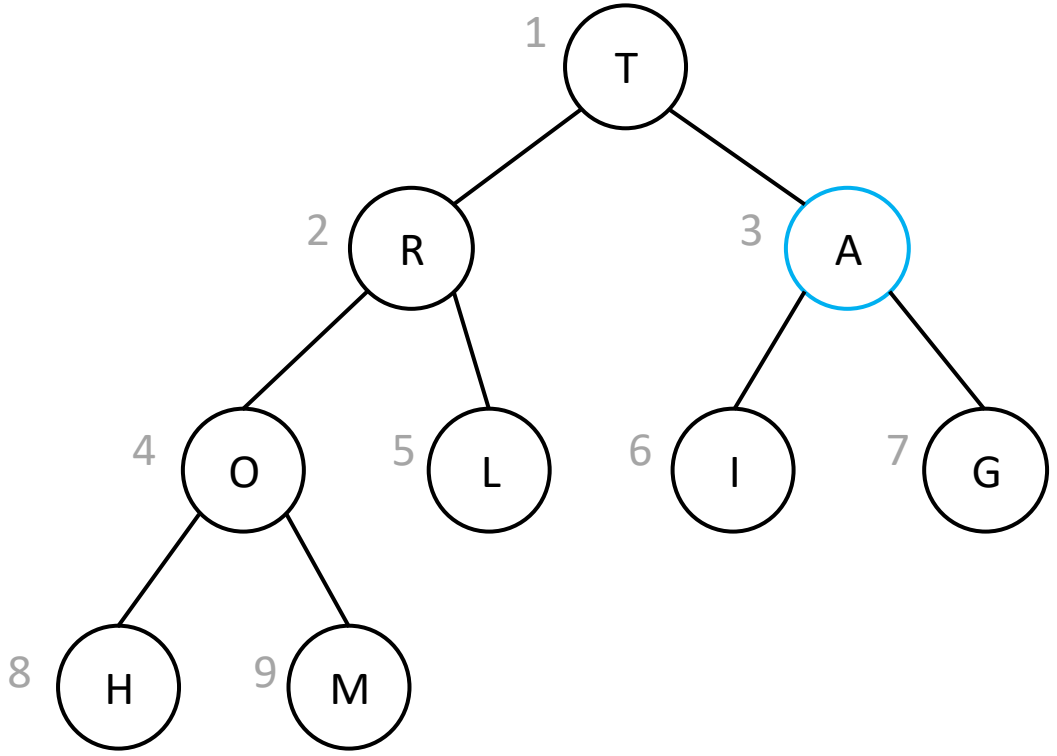
i



0	1	2	3	4	5	6	7	8	9
$*$	T	R	A	O	L	I	G	H	M

bubble-down(1)

Need to bubble-down further.

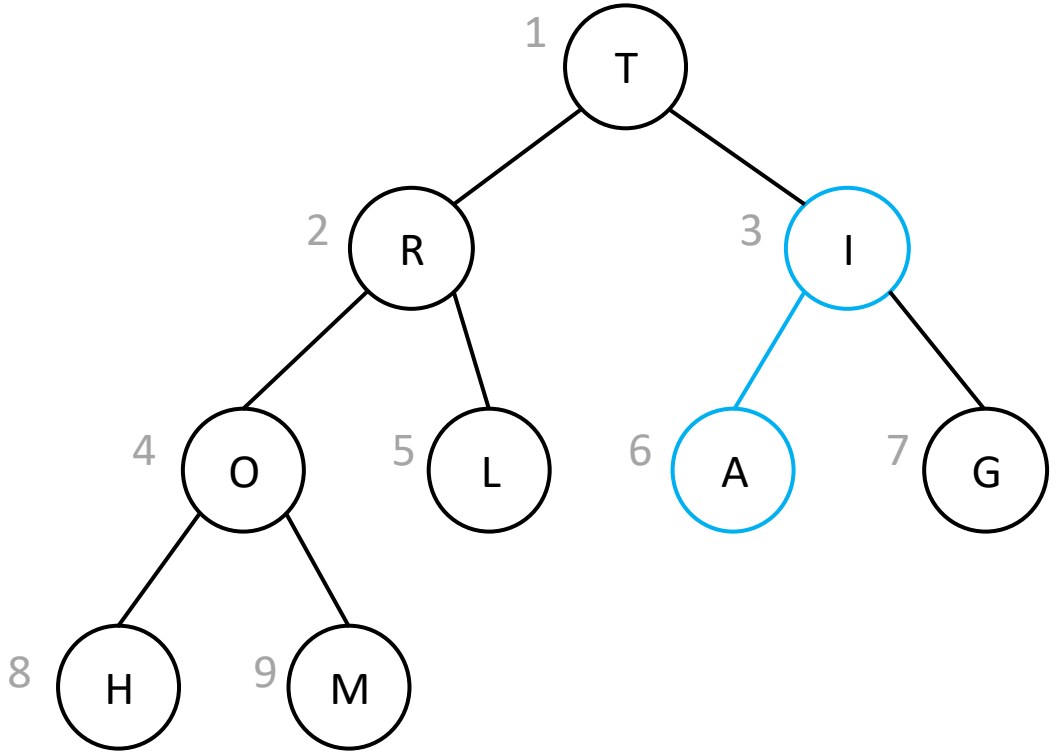


Problem 3. Make a max-heap out of the keys A, L, G, O, R, I, T, H, M, using the bottom-up algorithm.

- for $i = n/2, \ldots, 1$, invoke *bubble-down*(i);

i

0	1	2	3	4	5	6	7	8	9
$*$	T	R	I	O	L	A	G	H	M



bubble-down(1)

I is the max out of $\{A, I, G\}$.

So swap A and I.

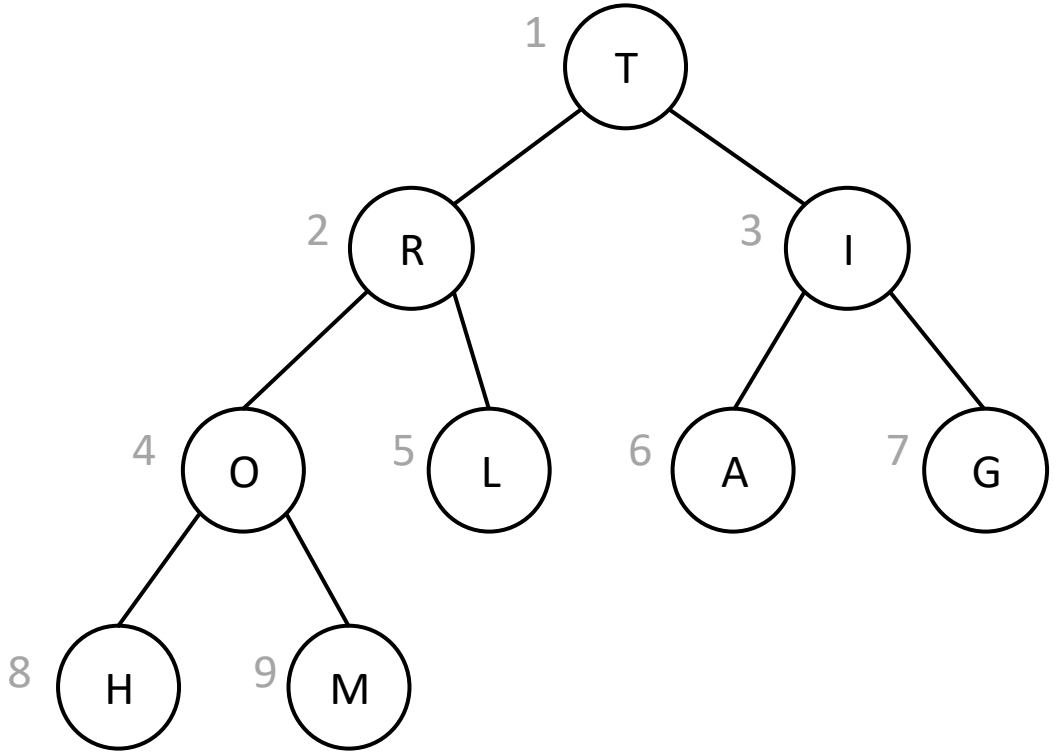
Problem 3. Make a max-heap out of the keys A, L, G, O, R, I, T, H, M, using the bottom-up algorithm.

- for $i = n/2, \ldots, 1$, invoke *bubble-down*(i);

0 1 2 3 4 5 6 7 8 9

$[*, T, R, I, O, L, A, G, H, M]$

We are done!



Tutorial Questions

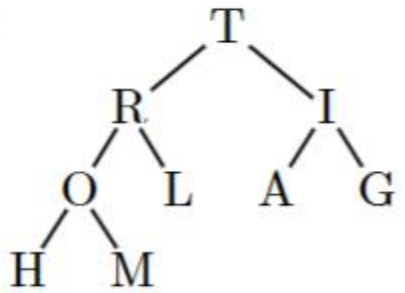
56. Make a max-heap out of the keys A, L, G, O, R, I, T, H, M, using the bottom-up algorithm.



Turning $H[1]\ldots H[n]$ into a Heap

```
for i ← ⌊n/2⌋ downto 1 do
    k ← i
    v ← H[k]
    heap ← False
    while not heap and 2 × k ≤ n do
        j ← 2 × k                          ▷ j is k's left child
        if j < n then
            if H[j] < H[j + 1] then
                j ← j + 1                  ▷ j is k's largest child
        if v ≥ H[j] then
            heap ← True
        else                               ▷ Promote H[j]
            H[k] ← H[j]
            k ← j
    H[k] ← v
```

Answer:



Or, as an array: T, R, I, O, L, A, G, H, M.

Tutorial Questions

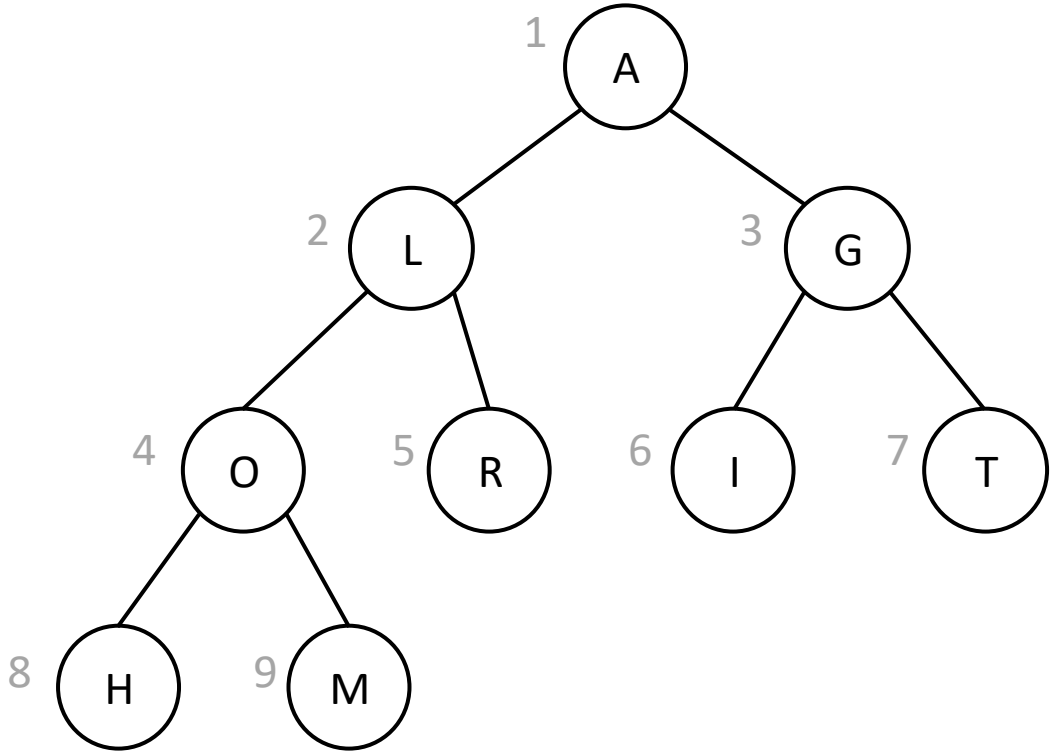
57. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

For $i = 2, \ldots, n$, invoke *bubble-up*(i).

0 1 2 3 4 5 6 7 8 9

$[*, A, L, G, O, R, I, T, H, M]$



Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

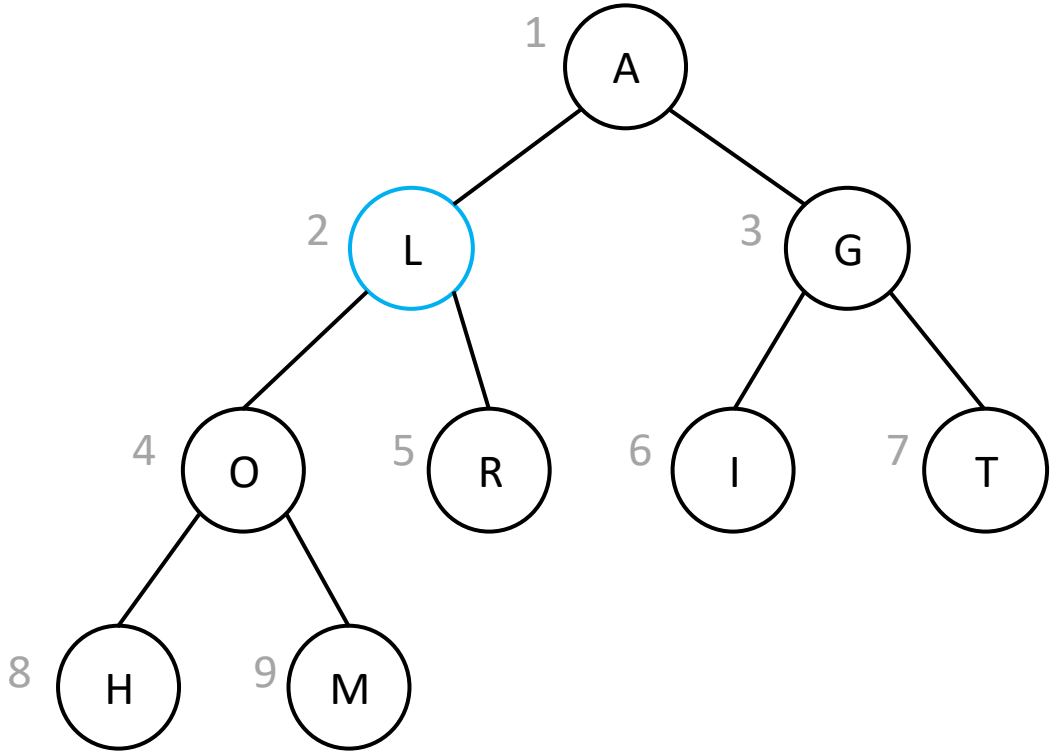
For $i = 2, \ldots, n$, invoke *bubble-up*(i).

i



0	1	2	3	4	5	6	7	8	9
$*$	A	L	G	O	R	I	T	H	M

bubble-up(2)

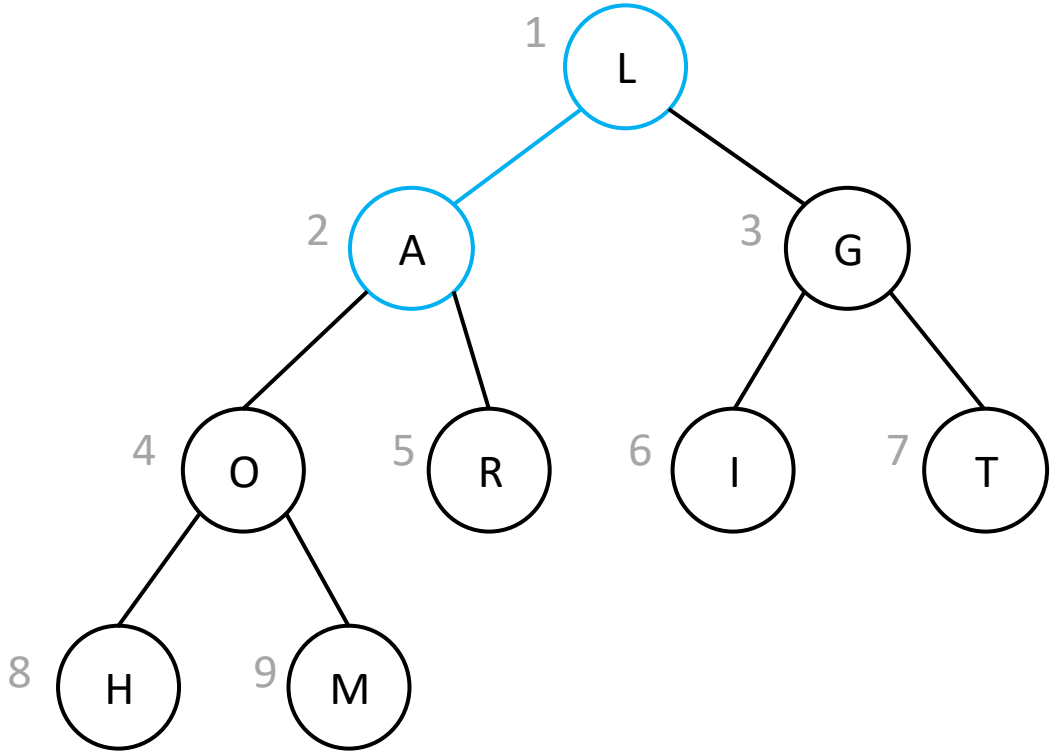


Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

For $i = 2, \ldots, n$, invoke *bubble-up*(i).

i

0	1	2	3	4	5	6	7	8	9
$*$	L	A	G	O	R	I	T	H	M



bubble-up(2)

L is larger than A.

So swap L and A.

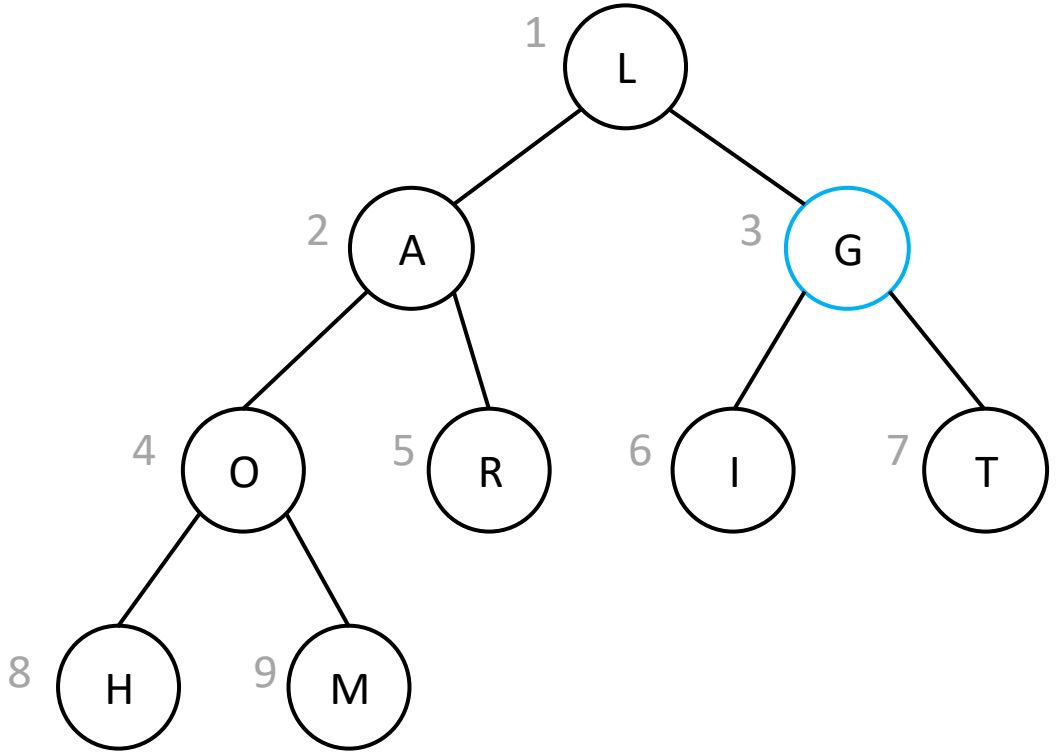
Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

For $i = 2, \ldots, n$, invoke *bubble-up*(i).

i

0	1	2	3	4	5	6	7	8	9
$*$	L	A	G	O	R	I	T	H	M

bubble-up(3)



Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

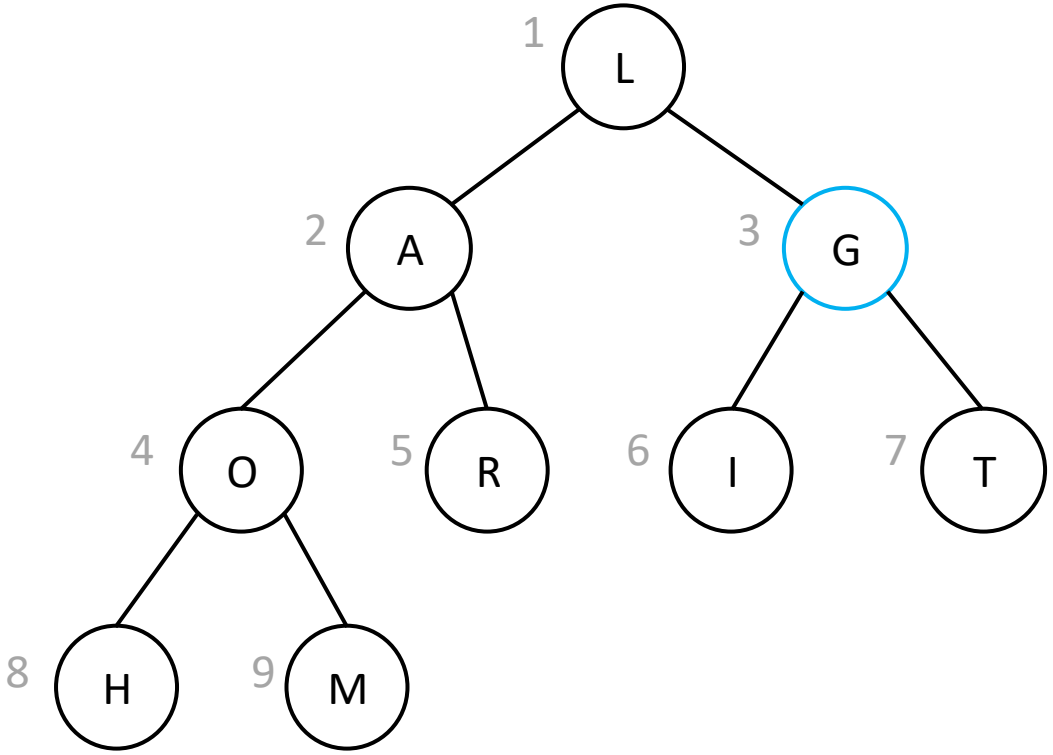
For $i = 2, \ldots, n$, invoke *bubble-up*(i).

i

0	1	2	3	4	5	6	7	8	9
$*$	L	A	G	O	R	I	T	H	M

bubble-up(3)

G is already smaller than L.



Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

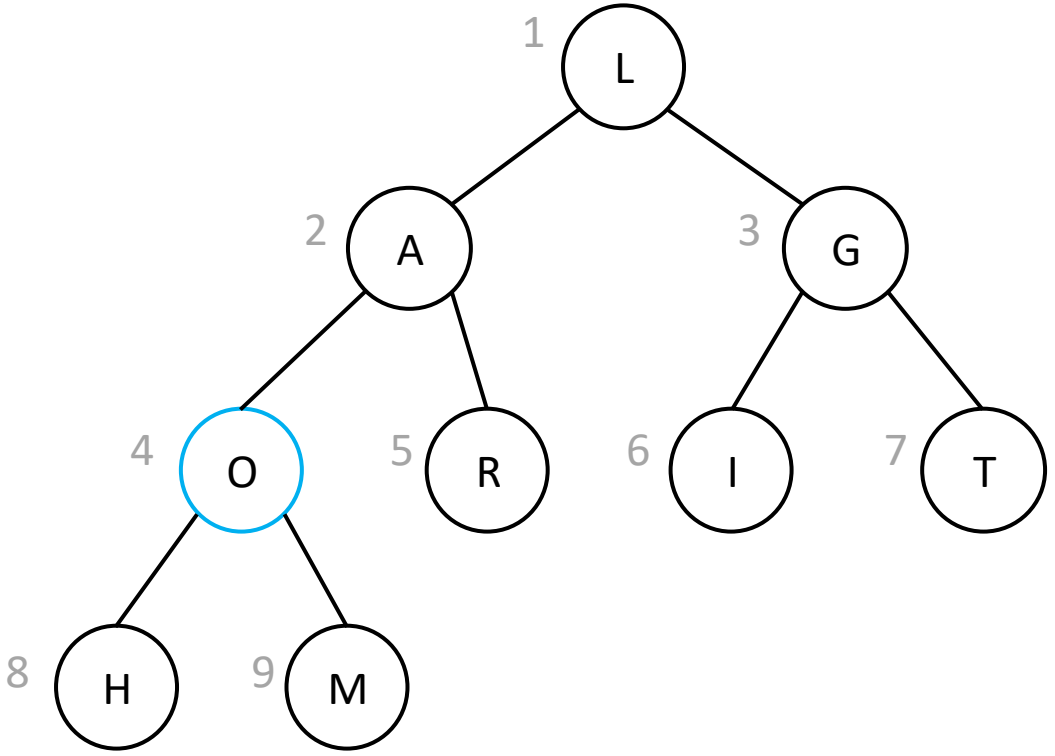
For $i = 2, \ldots, n$, invoke *bubble-up*(i).

i



0	1	2	3	4	5	6	7	8	9
$*$	L	A	G	O	R	I	T	H	M

bubble-up(4)



Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

For $i = 2, \ldots, n$, invoke *bubble-up*(i).

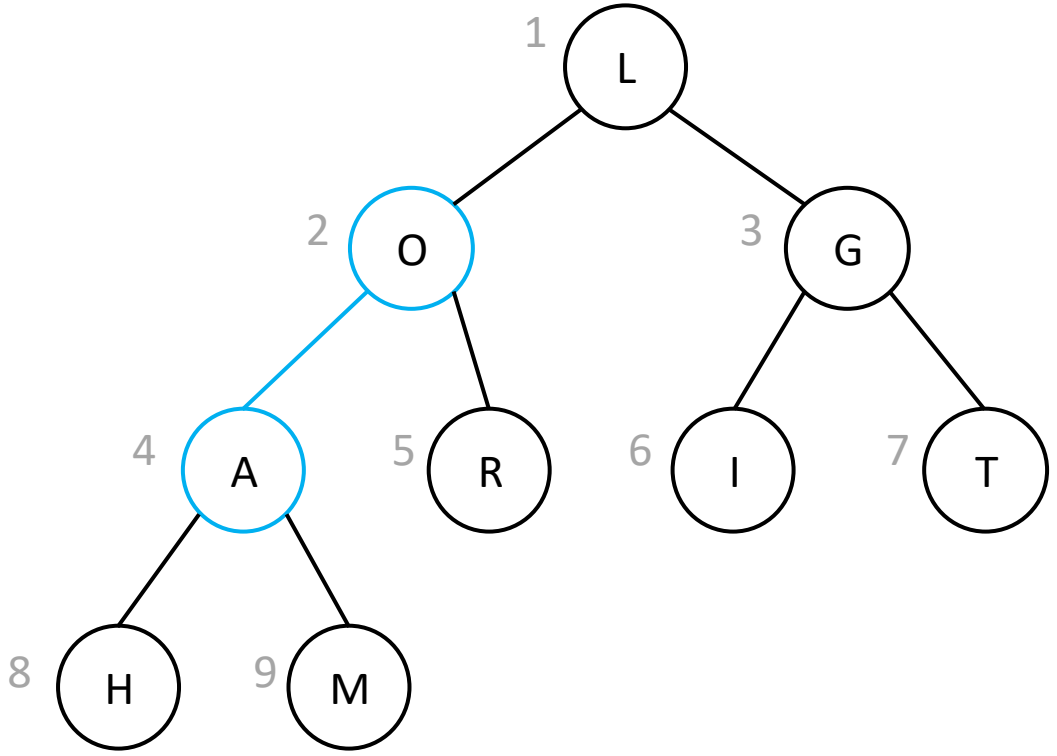
i

0	1	2	3	4	5	6	7	8	9
$*$	L	O	G	A	R	I	T	H	M

bubble-up(4)

O is larger than A.

So swap O with A.



Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

For $i = 2, \ldots, n$, invoke *bubble-up*(i).

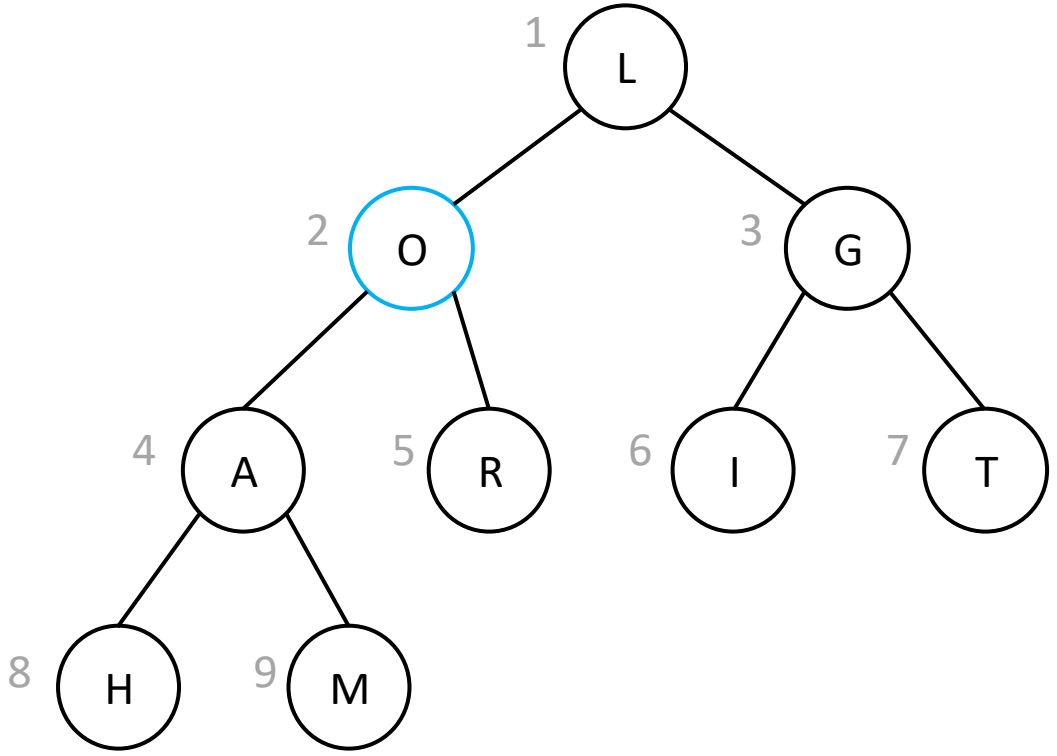
i



0	1	2	3	4	5	6	7	8	9
$*$	L	O	G	A	R	I	T	H	M

bubble-up(4)

Need to bubble-up further.



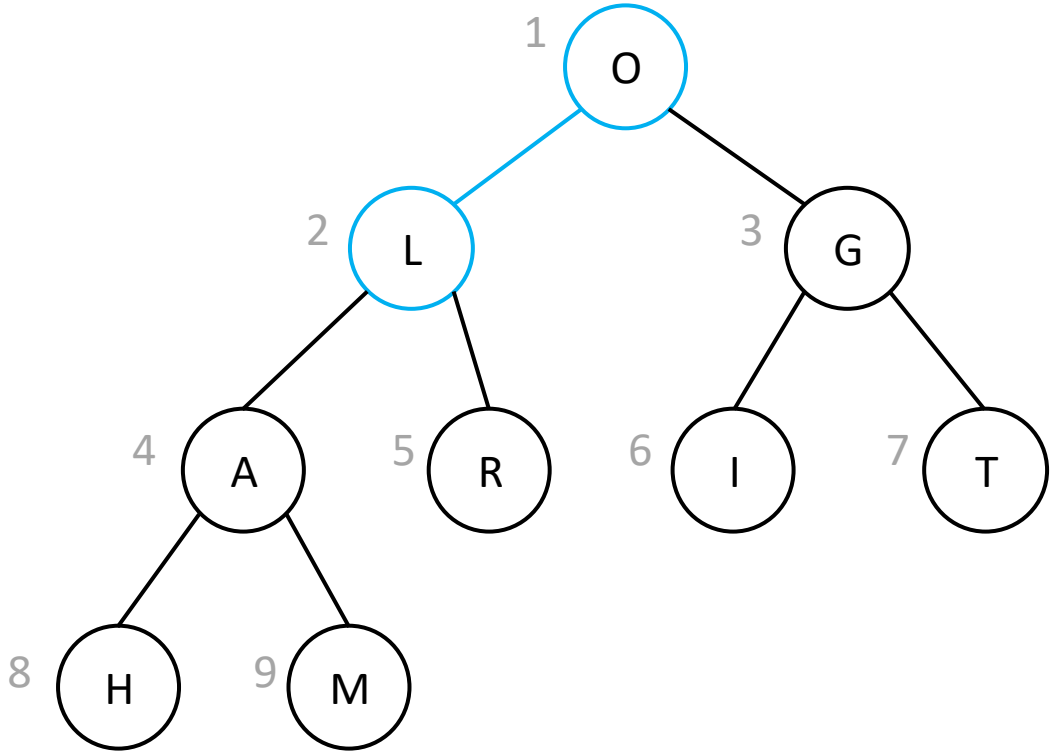
Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

For $i = 2, \ldots, n$, invoke *bubble-up*(i).

i



0	1	2	3	4	5	6	7	8	9
$*$	O	L	G	A	R	I	T	H	M



bubble-up(4)

O is larger than L.

So swap O with L.

Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

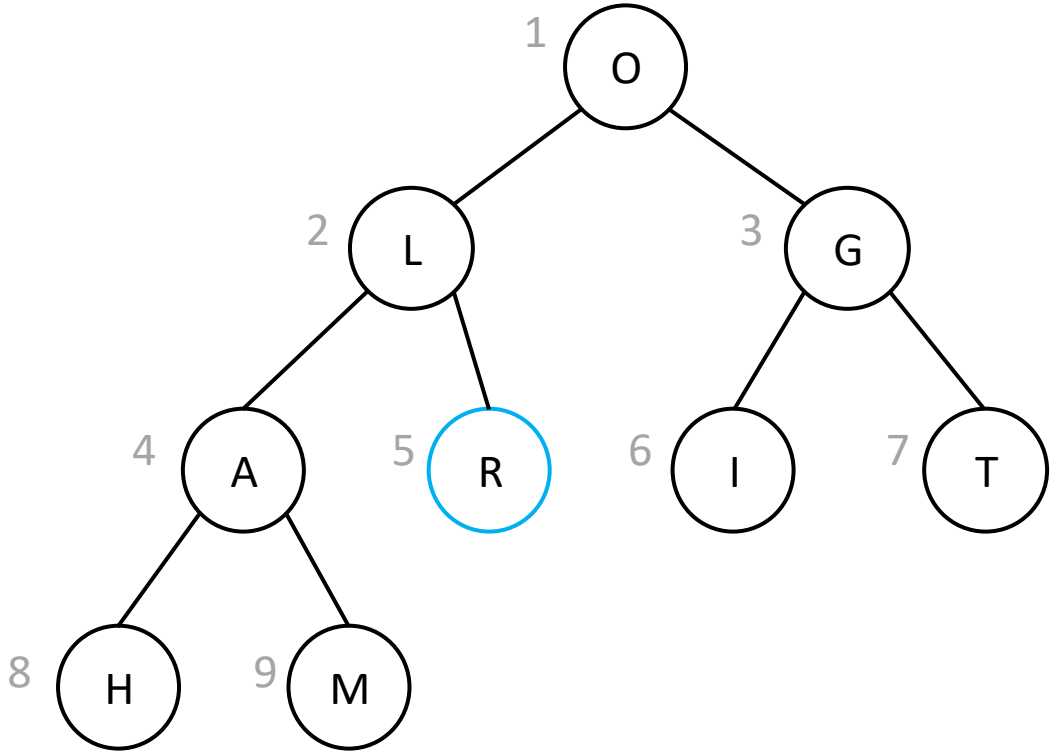
For $i = 2, \ldots, n$, invoke *bubble-up*(i).

i

0 1 2 3 4 5 6 7 8 9

$[*, O, L, G, A, R, I, T, H, M]$

bubble-up(5)



Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

For $i = 2, \ldots, n$, invoke *bubble-up*(i).

i

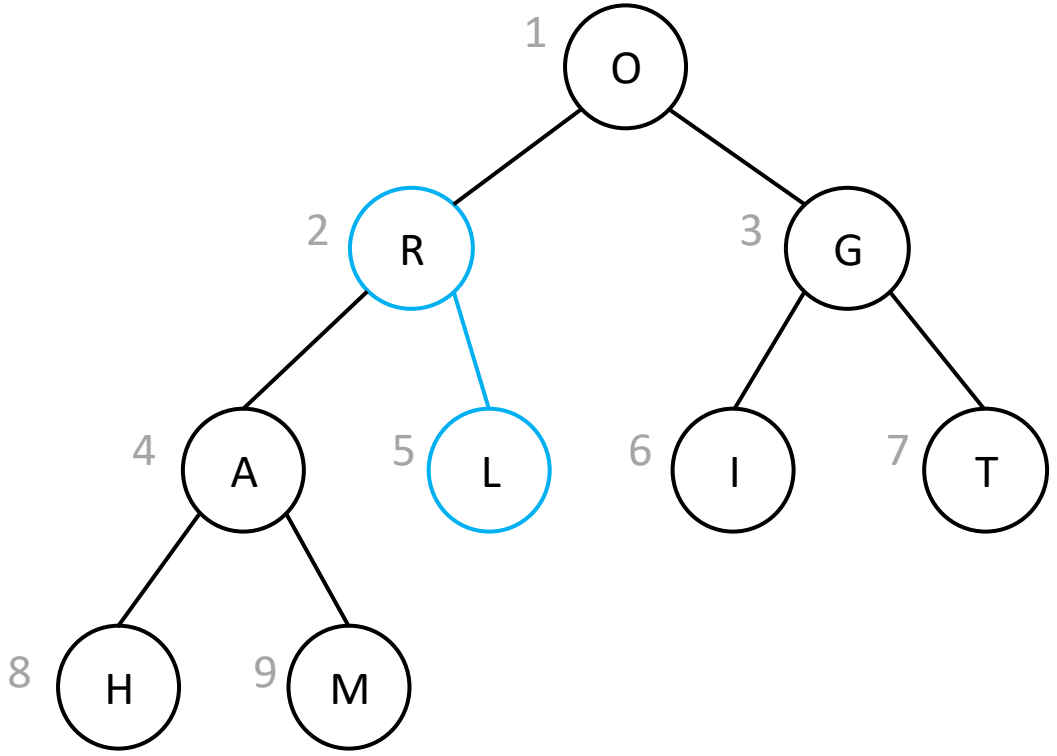
0 1 2 3 4 5 6 7 8 9

$[*, O, R, G, A, L, I, T, H, M]$

bubble-up(5)

R is larger than L.

So swap R with L.

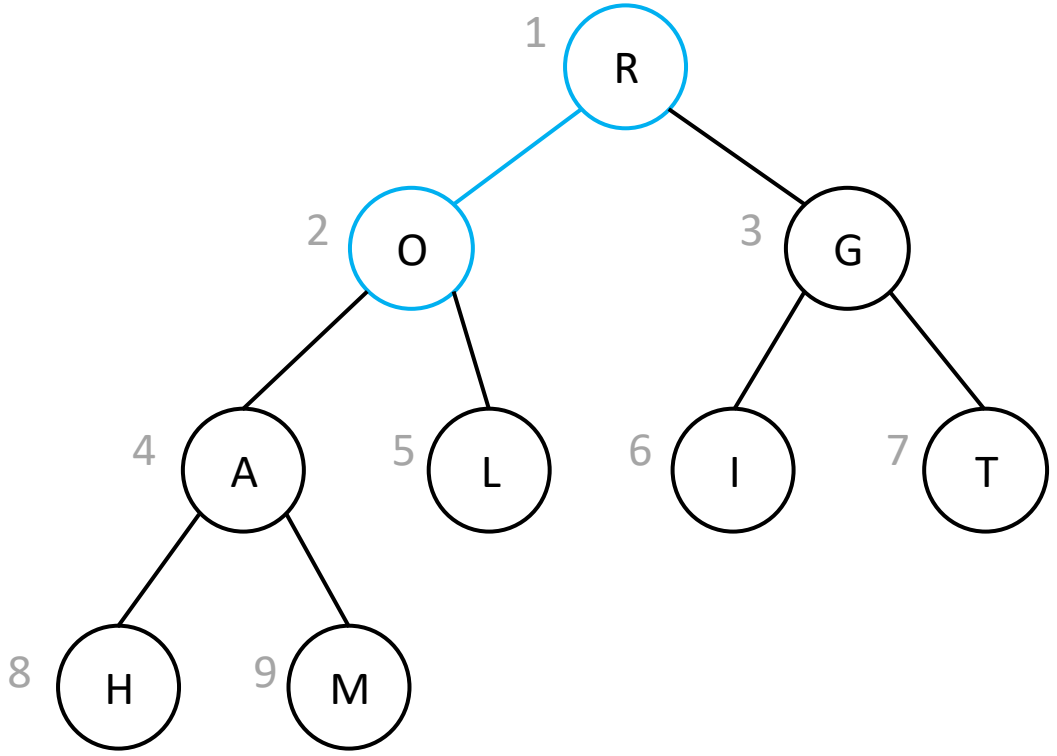


Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

For $i = 2, \ldots, n$, invoke *bubble-up*(i).

i

0	1	2	3	4	5	6	7	8	9
$*$	R	O	G	A	L	I	T	H	M



bubble-up(5)

R is larger than O.

So swap R with O.

Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

For $i = 2, \ldots, n$, invoke *bubble-up*(i).

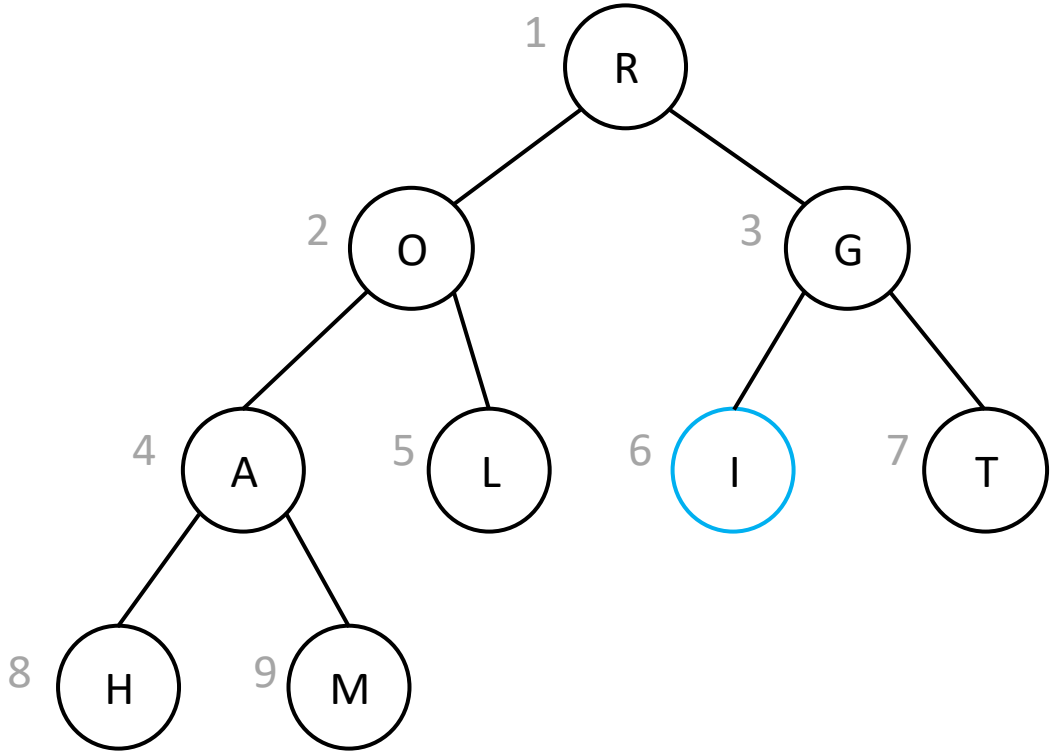
i



0 1 2 3 4 5 6 7 8 9

$[*, R, O, G, A, L, I, T, H, M]$

bubble-up(6)



Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

For $i = 2, \ldots, n$, invoke *bubble-up*(i).

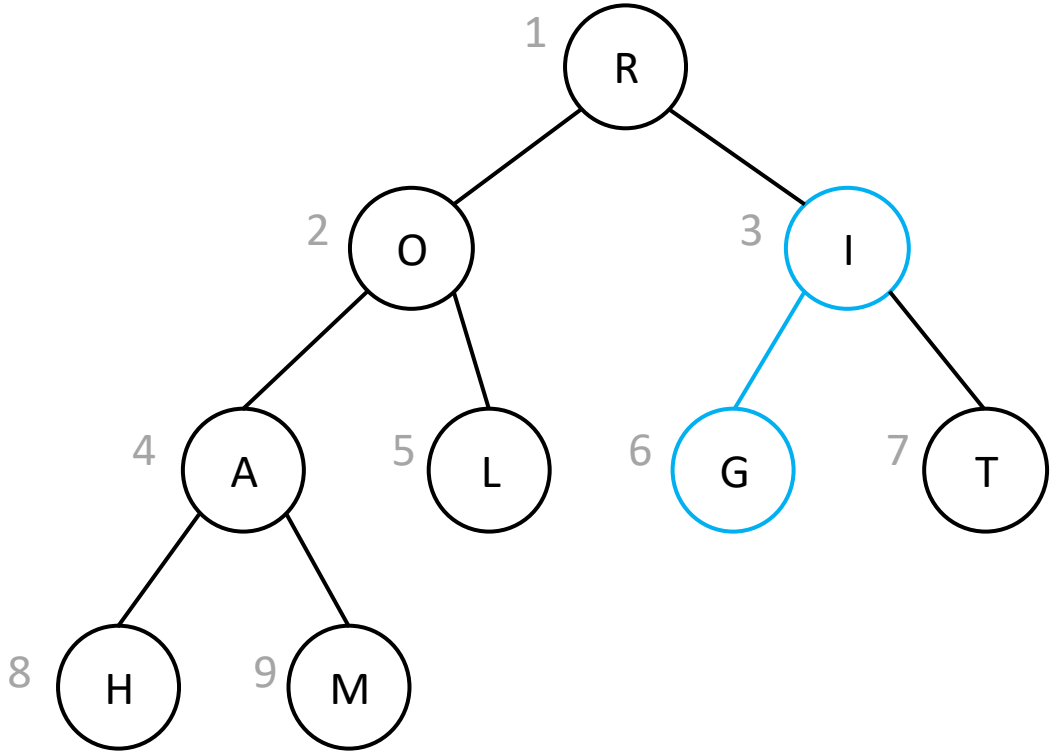
i

0	1	2	3	4	5	6	7	8	9
$*$	R	O	I	A	L	G	T	H	M

bubble-up(6)

I is larger than G.

So swap I with G.



Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

For $i = 2, \ldots, n$, invoke *bubble-up*(i).

i

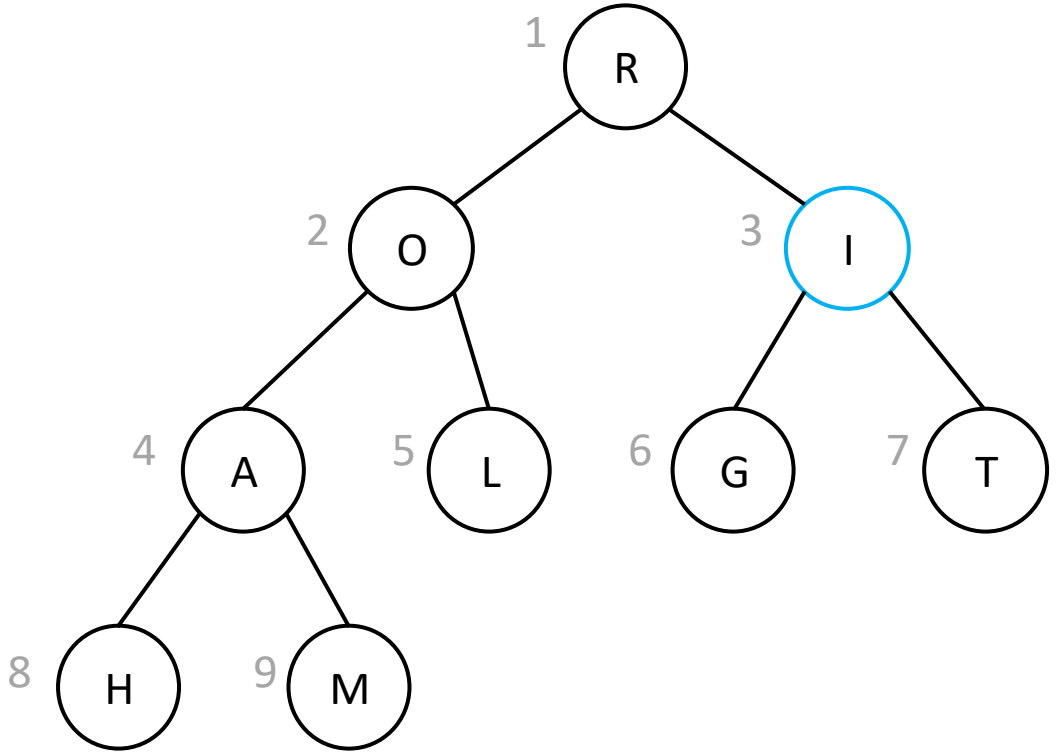


0 1 2 3 4 5 6 7 8 9

$[*, R, O, I, A, L, G, T, H, M]$

bubble-up(6)

I is already smaller than R.



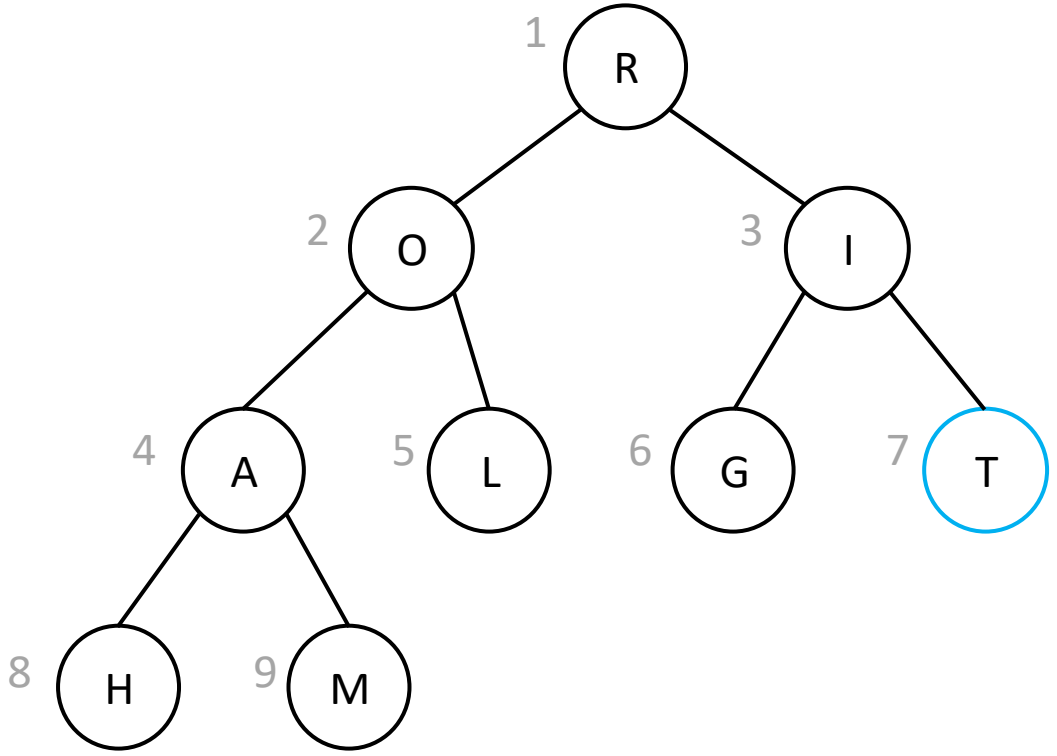
Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

For $i = 2, \ldots, n$, invoke *bubble-up*(i).

i

0	1	2	3	4	5	6	7	8	9
$*$	R	O	I	A	L	G	T	H	M

bubble-up(7)



Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

For $i = 2, \ldots, n$, invoke *bubble-up*(i).

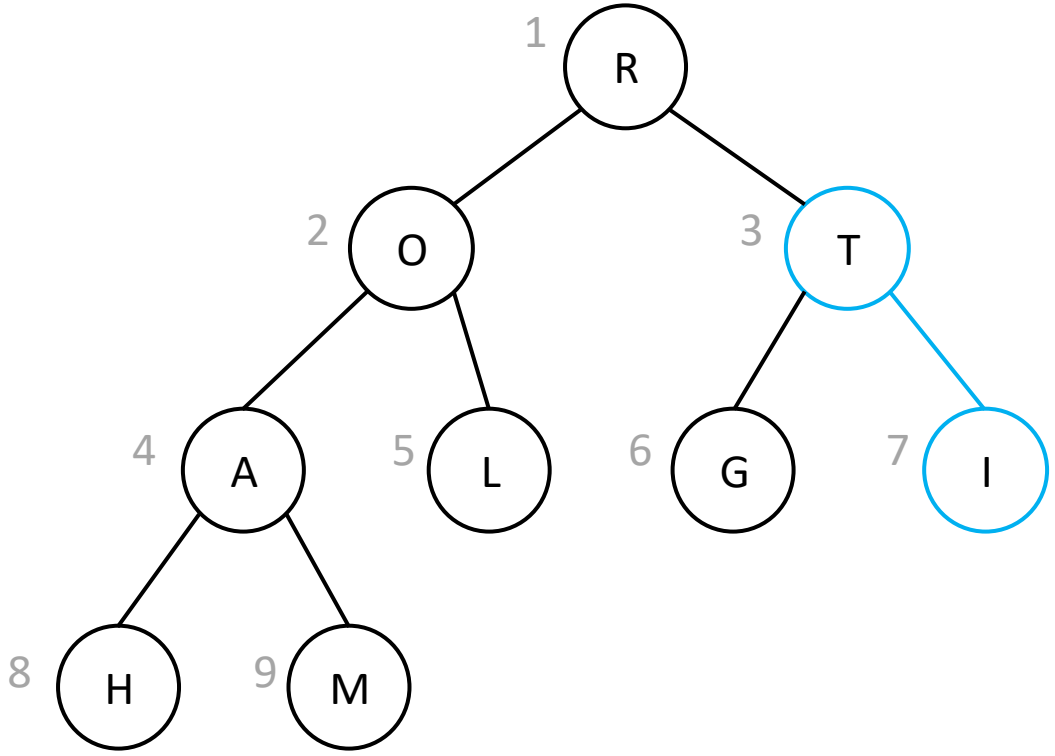
i

0	1	2	3	4	5	6	7	8	9
$*$	R	O	T	A	L	G	I	H	M

bubble-up(7)

T is larger than I.

So swap T with I.



Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

For $i = 2, \ldots, n$, invoke *bubble-up*(i).

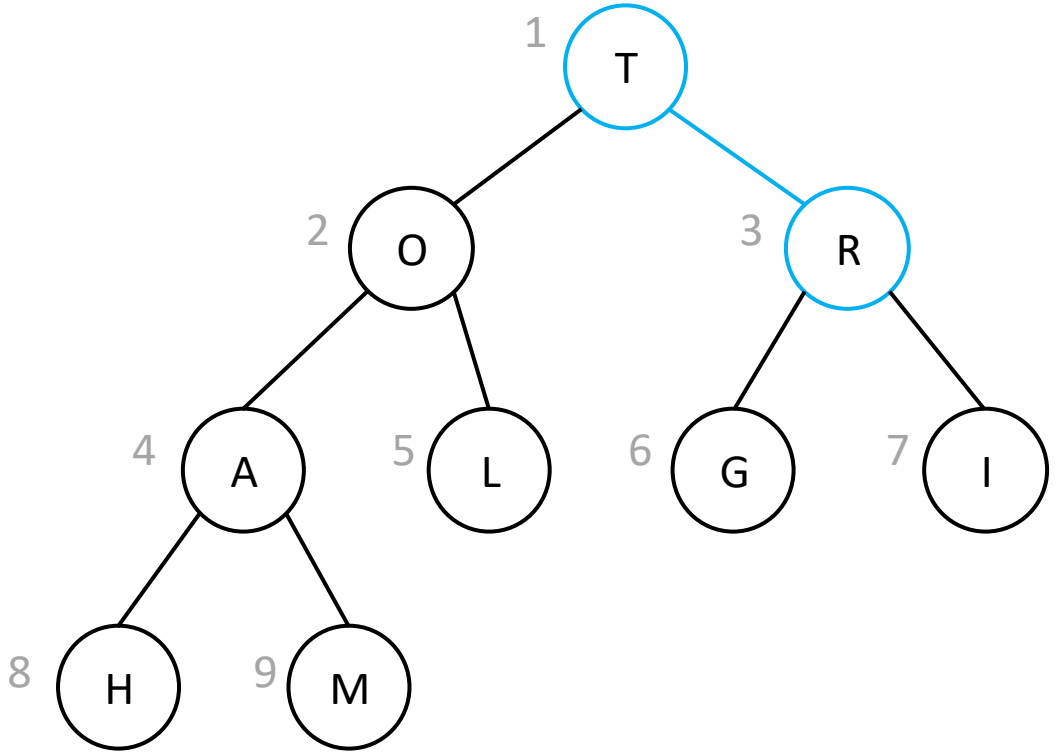
i

0	1	2	3	4	5	6	7	8	9
$*$	T	O	R	A	L	G	I	H	M

bubble-up(7)

T is larger than R.

So swap T with R.



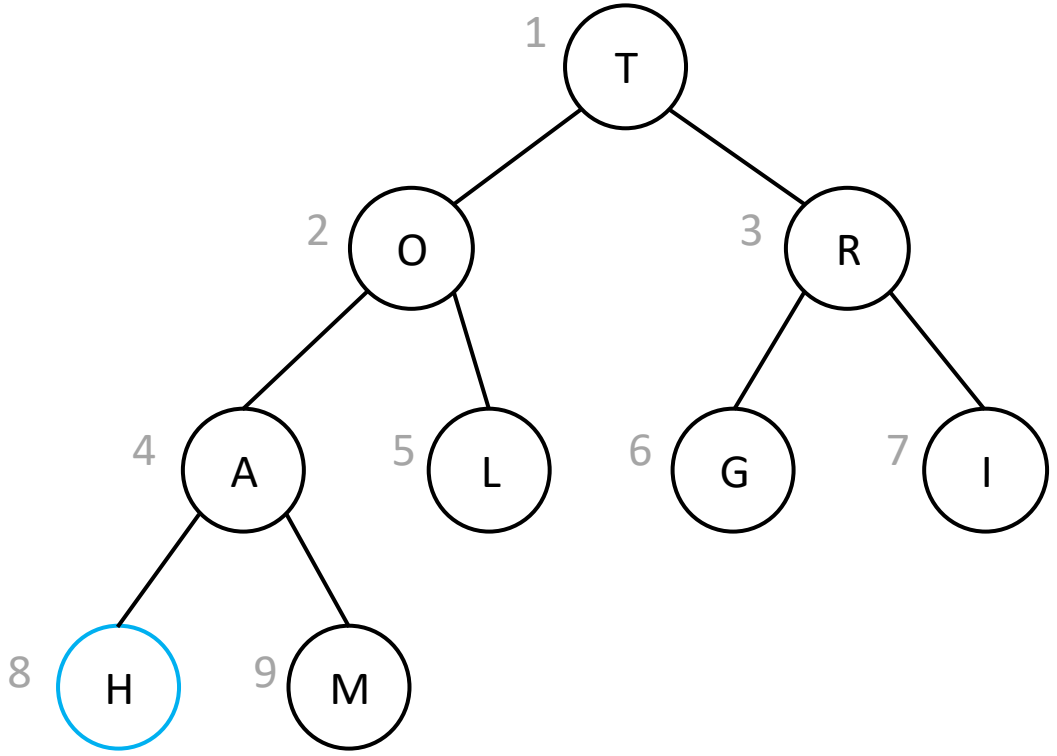
Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

For $i = 2, \ldots, n$, invoke *bubble-up*(i).

i

0	1	2	3	4	5	6	7	8	9
$*$	T	O	R	A	L	G	I	H	M

bubble-up(8)



Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

For $i = 2, \ldots, n$, invoke *bubble-up*(i).

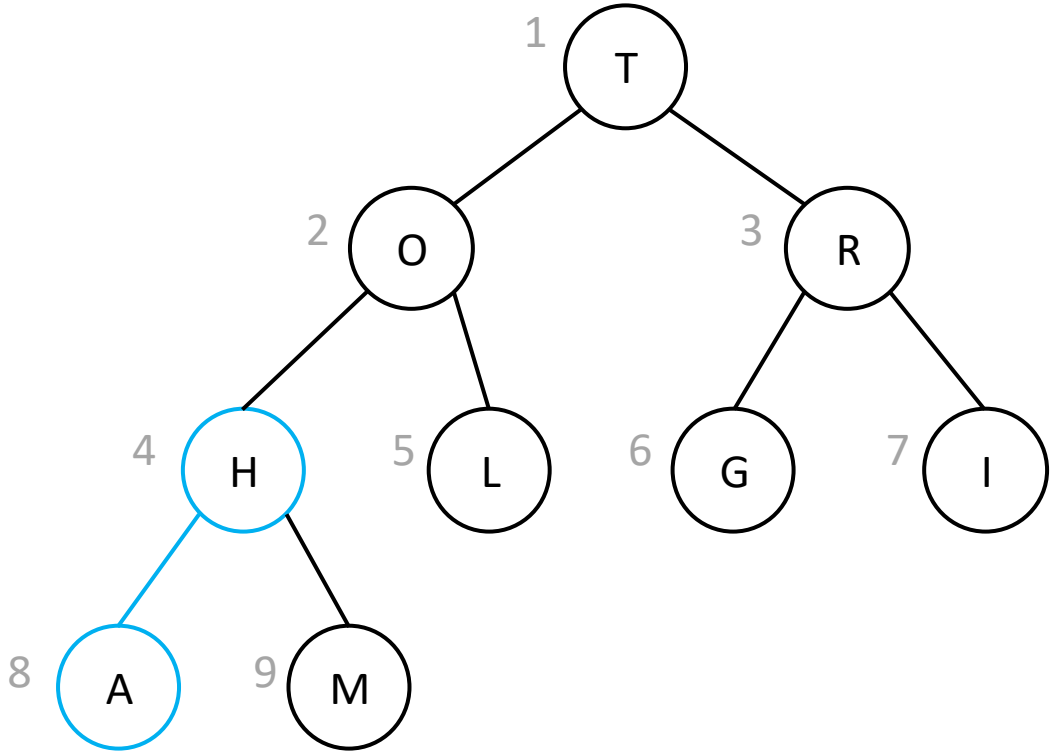
i

0	1	2	3	4	5	6	7	8	9
$*$	T	O	R	H	L	G	I	A	M

bubble-up(8)

H is larger than A.

So swap H with A.



Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

For $i = 2, \ldots, n$, invoke *bubble-up*(i).

i

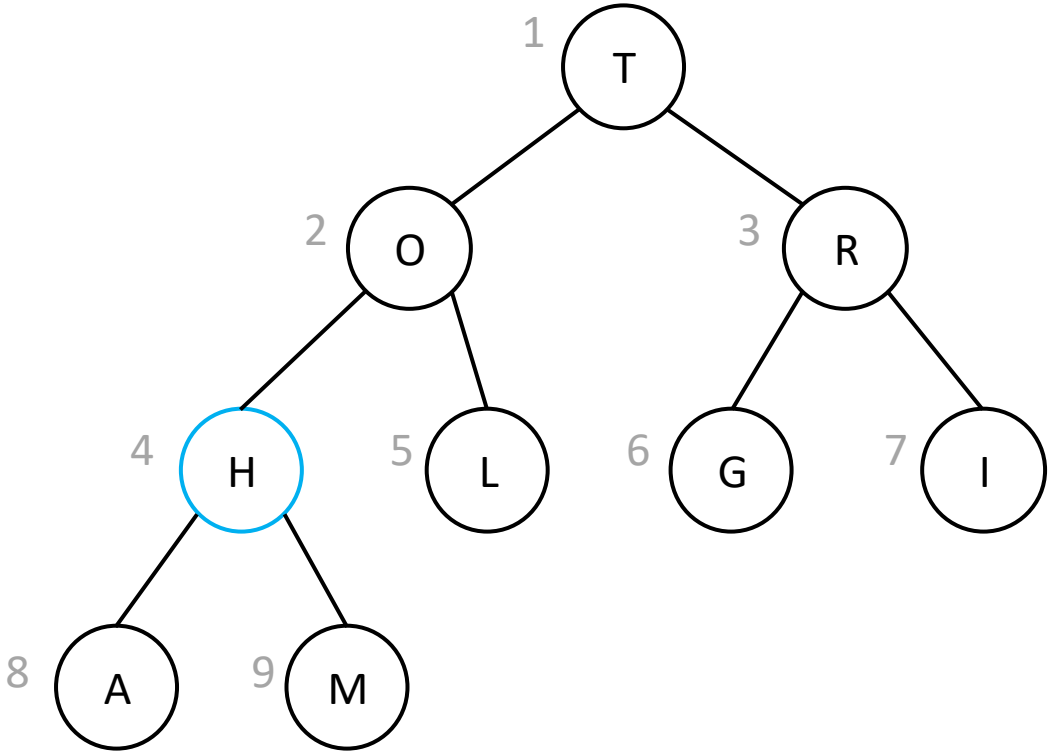


0 1 2 3 4 5 6 7 8 9

$[*, T, O, R, H, L, G, I, A, M]$

bubble-up(8)

H is already smaller than O.



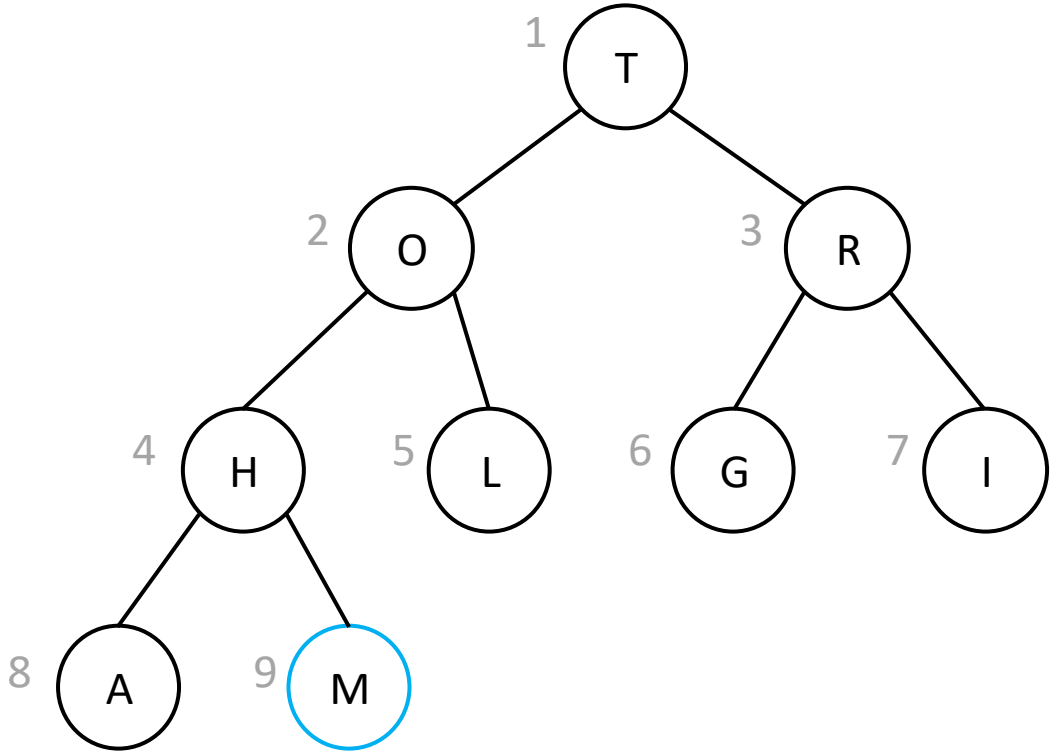
Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

For $i = 2, \ldots, n$, invoke *bubble-up*(i).

i

0	1	2	3	4	5	6	7	8	9
$*$	T	O	R	H	L	G	I	A	M

bubble-up(9)



Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

For $i = 2, \ldots, n$, invoke *bubble-up*(i).

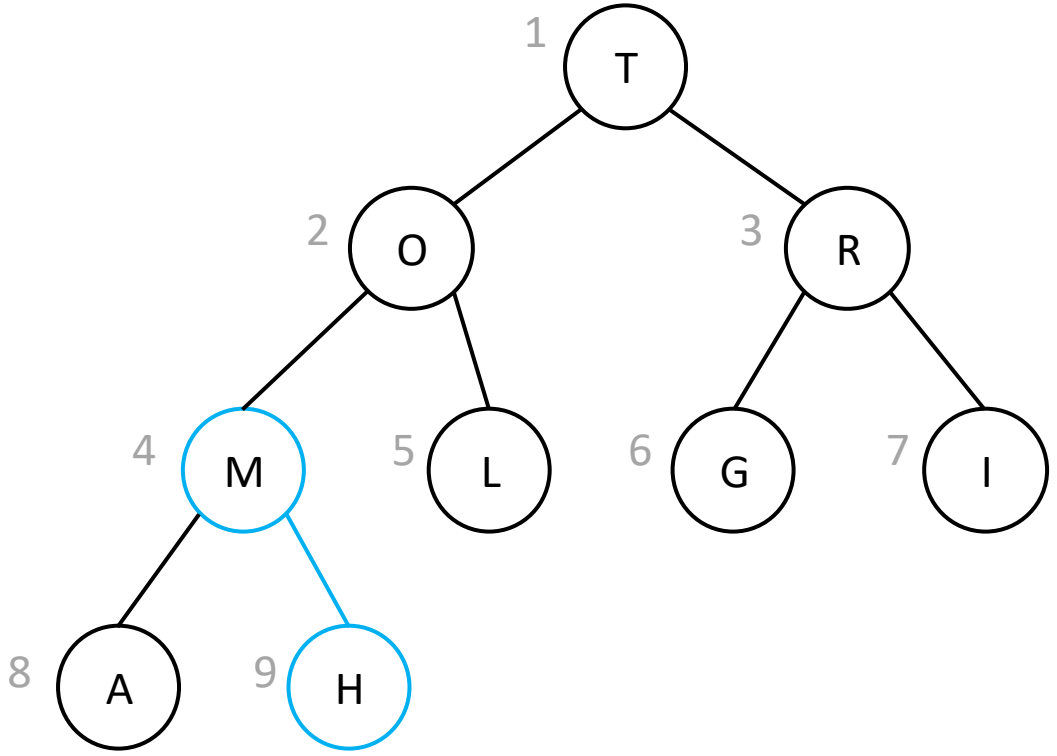
i

0	1	2	3	4	5	6	7	8	9
$*$	T	O	R	M	L	G	I	A	H

bubble-up(9)

M is larger than H.

So swap M with H.



Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

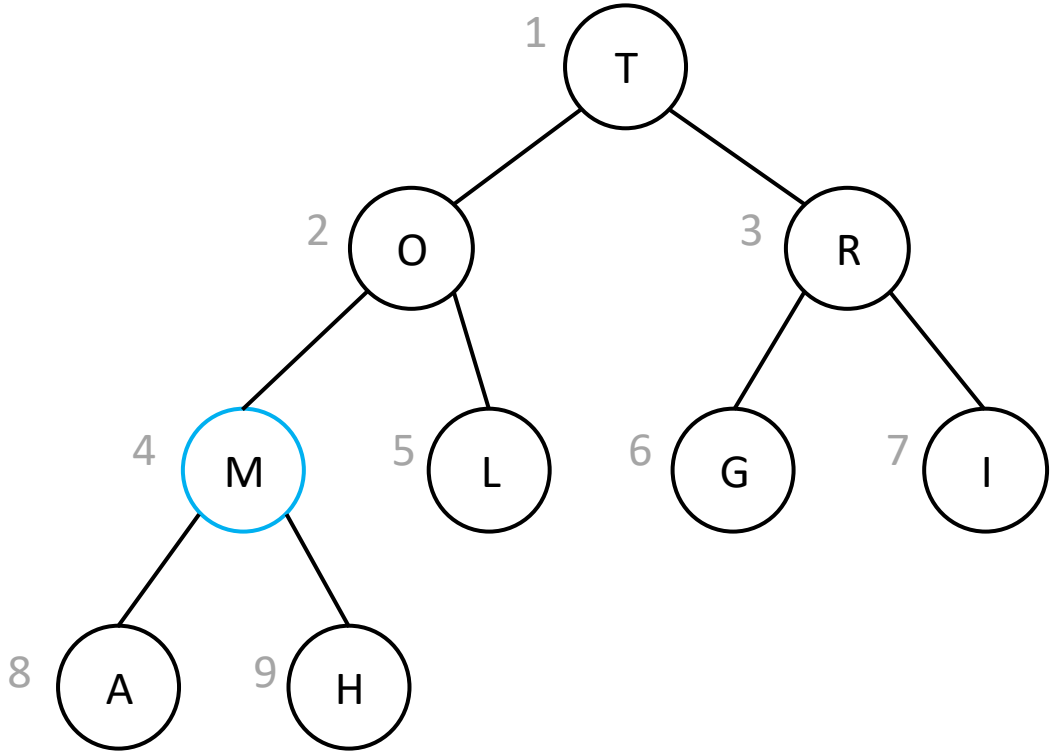
For $i = 2, \ldots, n$, invoke *bubble-up*(i).

i

0	1	2	3	4	5	6	7	8	9
$*$	T	O	R	M	L	G	I	A	H

bubble-up(9)

M is already smaller than O.

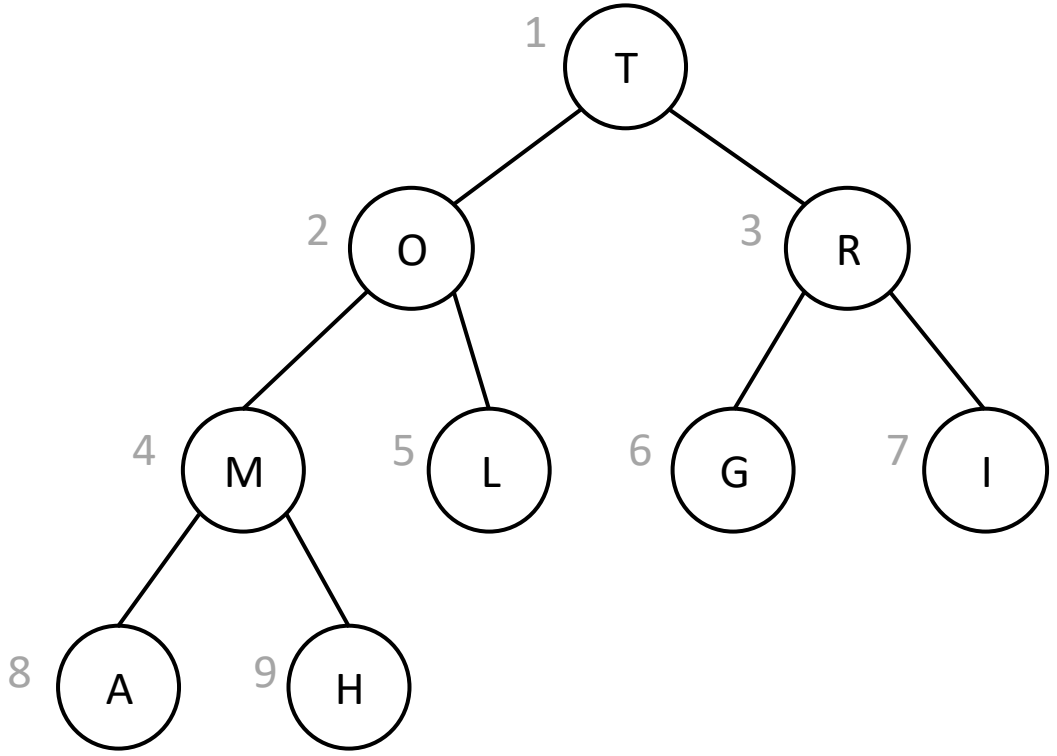


Problem 4. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

0 1 2 3 4 5 6 7 8 9

$[*, T, O, R, M, L, G, I, A, H]$

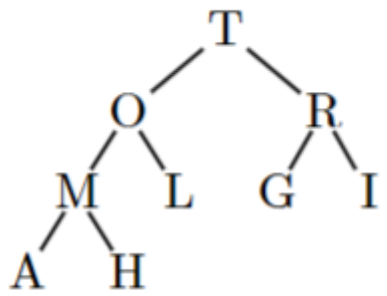
We are done!



Tutorial Questions

57. Construct a max-heap from the empty heap by inserting the keys A, L, G, O, R, I, T, H, M, one by one, in that order. Is the result the same as the heap from the previous question?

Answer:



Or, as an array: T, O, R, M, L, G, I, A, H, which is different from the previous question's answer.

Tutorial Questions

58. Give an algorithm for deciding whether an array $A[1]..A[n]$ is a heap.

A heap is a complete binary tree which satisfies the heap condition:

Each child has a priority (key) which is no greater than its parent's.

Tutorial Questions

58. Give an algorithm for deciding whether an array $A[1]..A[n]$ is a heap.

```
function IsHeap(A[·], n)
    for i ← 2 to n do
        if A[i] > A[i/2] then
            return False
    return True
```

Tutorial Questions

59. Apply the heapsort algorithm to A, L, G, O, R, I, T, H, M.

Heapsort is a $\Theta(n \log n)$ sorting algorithm.

Given an unsorted array $H[1] \ldots H[n]$:

Step 1 Turn H into a heap.

Step 2 Apply the eject operation $n-1$ times.

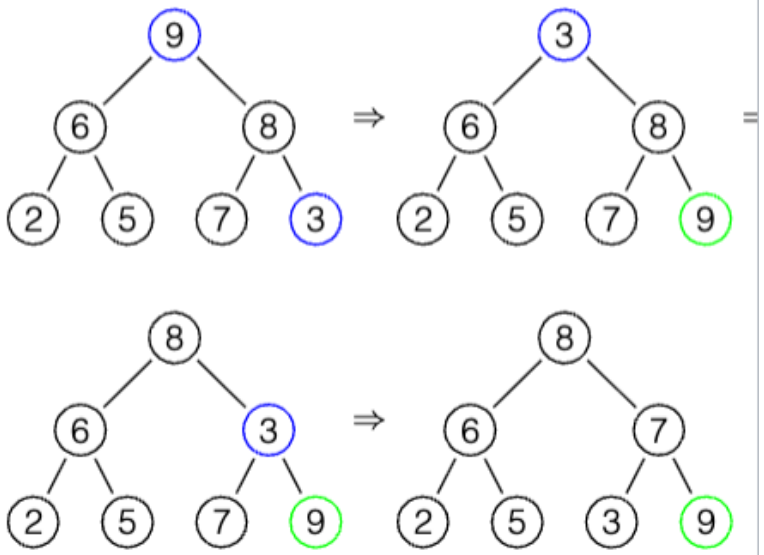
Ejecting a Maximal Element from a Heap

THE UNIVERSITY OF MELBOURNE

Here the idea is to swap the root with the last item z in the heap, and then let z "sift down" to its proper place.

After this, the last element (here shown in green) is no longer considered part of the heap, that is, n is decremented.

Ejection is $O(\log n)$.



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Thank you for coming!

See you all after the reading break! Relax ☺